

소프트웨어 공학

개인화 AI 닥터 서비스

2조

개인화 AI 닥터 서비스

01

프로젝트 개요

02

요구사항 정의 + 요구사항
분석

03

시스템 아키텍처

04

도메인 설계 (UI + API + 클
래스 + 시퀀스)

05

데이터 설계 (ERD)

06

개발 일정

07

추후 대응

문제 사항	솔루션
●의료 정보 관리의 부재 ○의료 정보가 흩어져 있어 관리가 힘들 ●의료 지식의 장벽 ○일반인이 해석하기엔 힘든 의료 지식 ●사후 관리의 부재 ○사후 건강 관리가 어려움	●세대, 성별 가리지 않고 사용하기 쉽고 깔끔한 인터페이스 웹 페이지 제공 ●LLM과 벡터 DB를 이용한 사용자 의료 데이터 기반 맞춤 대시보드 및 솔루션의 제공

프로젝트 개요

개발 목표

- ☑ 개인의 처방 내역과 일상 건강 기록을 통합 관리, 그것을 기반으로 AI가 개인에게 맞춘 분석을 제공한다.
- ☑ 사용자가 이해하기 쉽고 빠르게 눈으로 건강 상태를 확인할 수 있게 한다.
- ☑ RAM 2GB 서버 기준 평균 초당 5명 내외의 AI API호출을 원활히 처리할 수 있어야 한다 (MVP 기준, DB 2GB)

프로젝트 참여 인원

역할	이름	담당 파트	비고
팀장	박수한	● DB 설계 및 관리 ● API 개발 ● AI 비즈니스 로직 개발	
팀원	남주연	● 인증/회원 API 개발 ● Sping Security 구현 ● 회원 비즈니스 로직 개발	
팀원	서문창	● 처방전 API 개발 ● 처방전 도메인 DB 설계 ● 처방전 비즈니스 로직 개발	
팀원	바트빌렉아노진	● API 개발	

프로젝트 참여 인원

역할	이름	담당 파트	비고
팀원	옹흐서드아노진	• API 개발	
팀원	신찬욱	• 웹 담당 • UI/UX 설계 • 프론트 개발담당	
팀원	김희민	• 웹 담당 • UI/UX 설계 • 프론트 개발담당	

요구사항 정의

요구사항 정의서

요구사항 정의

요구사항명	기능 ID	기능명	요구사항 내용
개요	-	시스템명	AI 기반 개인화 건강 관리(닥터) 서비스
	-	목적	사용자의 건강 이력 및 처방 정보를 기반으로 건강 상태를 분석하고 맞춤형 추천 및 시각화 정보를 제공한다
	-	대상 사용자	일반 사용자 및 관리자
인증 요구사항	FR-01	회원가입	사용자는 로그인 화면에서 이름, 전화번호, 이메일 등의 입력을 통한 회원가입을 할 수 있다
	FR-02	로그인	사용자는 로그인 화면에서 id, password 기반으로 로그인할 수 있다
	FR-03	로그아웃	사용자는 로그아웃 버튼을 통해 로그아웃할 수 있다
	FR-04	회원 정보 조회	사용자는 자신의 정보를 메인 화면 내 정보에서 조회할 수 있다
	FR-05	회원 정보 수정	사용자는 자신의 정보를 메인 화면 내 정보에서 수정할 수 있다
	FR-06	회원 탈퇴	사용자는 자신의 정보를 메인 화면 내 정보에서 비밀번호 인증 후 탈퇴할 수 있다

요구사항 정의

요구사항명	기능 ID	기능명	요구사항 내용
처방전 데이터 요구사항	FR-07	처방전 정보 입력	사용자는 나의 처방전→처방 정보 입력화면에서 직접입력을 통해 처방전정보를 입력할 수 있다
	FR-08	처방전 목록 조회	사용자는 나의 처방전→처방 목록 화면에서 자신의 처방전 목록을 조회할 수 있다
	FR-09	처방전 상세 조회	사용자는 나의 처방전→처방 상세 화면에서 특정 처방전의 상세 정보 및 처방약 목록을 조회할 수 있다
	FR-10	처방전 정보 수정	사용자는 나의 처방전→처방 정보 수정에서 이때까지의 처방 정보를 확인하고 수정할 수 있다
	FR-11	처방전 정보 삭제	사용자는 나의 처방전→처방 기록 삭제에서 이때까지의 처방 정보를 삭제할 수 있다
건강 이력 요구사항	FR-12	건강 이력 입력	사용자는 나의 건강→건강정보 입력 화면에서 자신의 건강 이력을 직접입력으로 입력할 수 있다
	FR-13	건강 이력 조회	사용자는 나의 건강→건강정보 목록 화면에서 자신의 건강 이력을 조회할 수 있다
	FR-14	건강 이력 수정	사용자는 나의 건강→건강정보 수정에서 자신의 건강 이력을 수정할 수 있다
	FR-15	건강 이력 삭제	사용자는 나의 건강→건강정보 수정에서 자신의 건강 이력을 삭제할 수 있다

요구사항 정의

요구사항명	기능 ID	기능명	요구사항 내용
AI 서비스 요구사항	FR-16	건강 상태 분석	시스템은 사용자 데이터를 기반으로 건강 상태를 분석할 수 있다
	FR-17	건강 관리 추천	시스템은 사용자에게 건강 관리 추천을 제공할 수 있다
	FR-18	데이터 시각화	시스템은 메인 화면에서 저장된 건강 데이터 통계와 솔루션을 시각화하여 제공할 수 있다

요구사항 분석

요구사항 분석

기술 스택



Amazon RDS



React JS



요구사항 분석

기능 분석

FR01~FR15 :로그인 / 회원가입 / CRUD 기능 : 일반적 웹 기능

FR16~FR18 : LLM API 호출을 통한 요약 제공 : AI 응답 기능

← 실질적 서비스 주요 병목.

비교 분석

일반 CRUD

DB connection pool

Slow Query

Disk I/O

VS

일반 CRUD+LLM 호출

Thread & connection
고갈

Network Latency

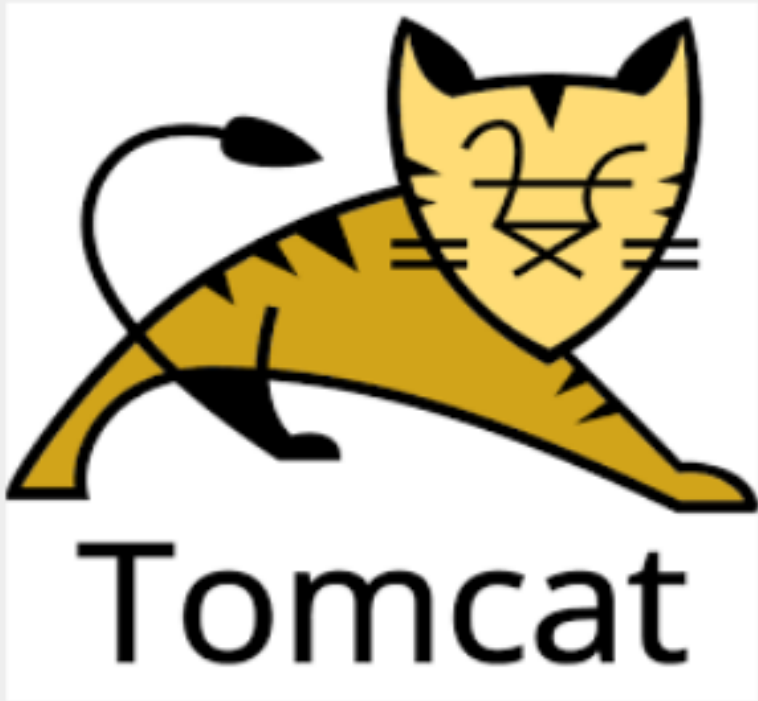
요구사항 분석

개발 스택 분석

서버 개발 언어 : JAVA Spring



WAS (Web Applicatoin Server): Tomcat



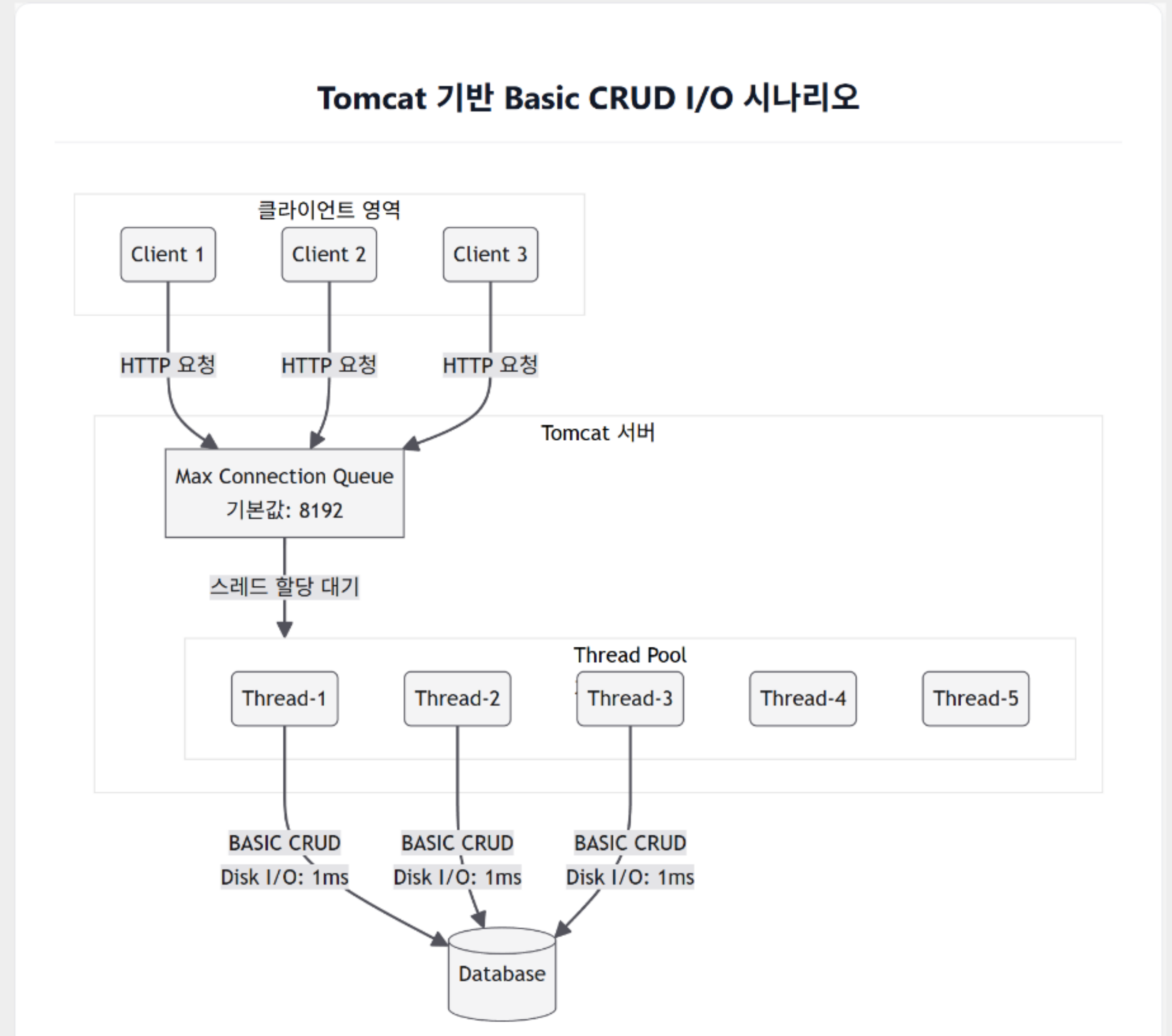
항목	계산	예상점유	비고
OS&JVM	기본 가동비 용	700MB	Linux 및 스프링 부트 라이브러리 적재
Thread Stack	Default(200)x1 MB	200MB	Tomcat Thread pool 호스팅 비용
Heap	CRUD : 10~30kb AI : 1~2MB	100 ~ 300MB	AI 요청/응답 JSON 및 DB CRUD Transaction 수행
Free	여유 20% 보장	200MB	서버 여유 공간 확보

요구사항 분석 - Tomcat's ThreadPool

요구사항 분석

일반적인 CRUD 기능의 경우:

주 로직 수행 대상인 DB 가 물리적으로 가까움
오랜 시간이 걸리지 않고 동기적 작업 수행 /
스레드 반환이 빨라 작업처리에 병목 X



요구사항 분석 - LLM API's Thread occupancy

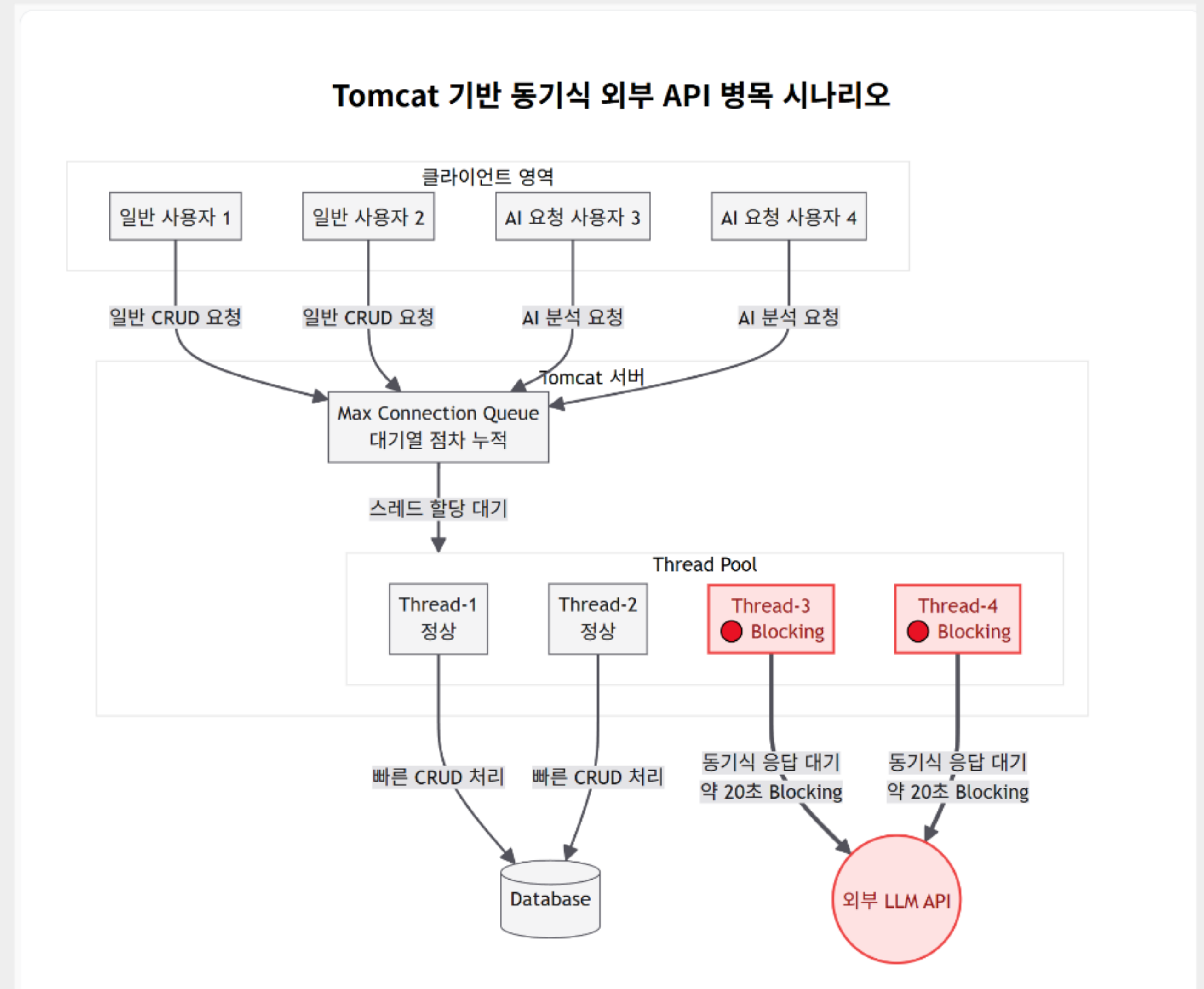
요구 사항 분석

LLM API 호출의 경우:

해당 작업이 동기적으로 수행될 경우 외부 LLM의 API 응답이 돌아올 때까지 Tomcat의 스레드는 Blocking.

추가 고려 요소

- Network Latency
- LLM API 서버의 상태
- DB Connection pool

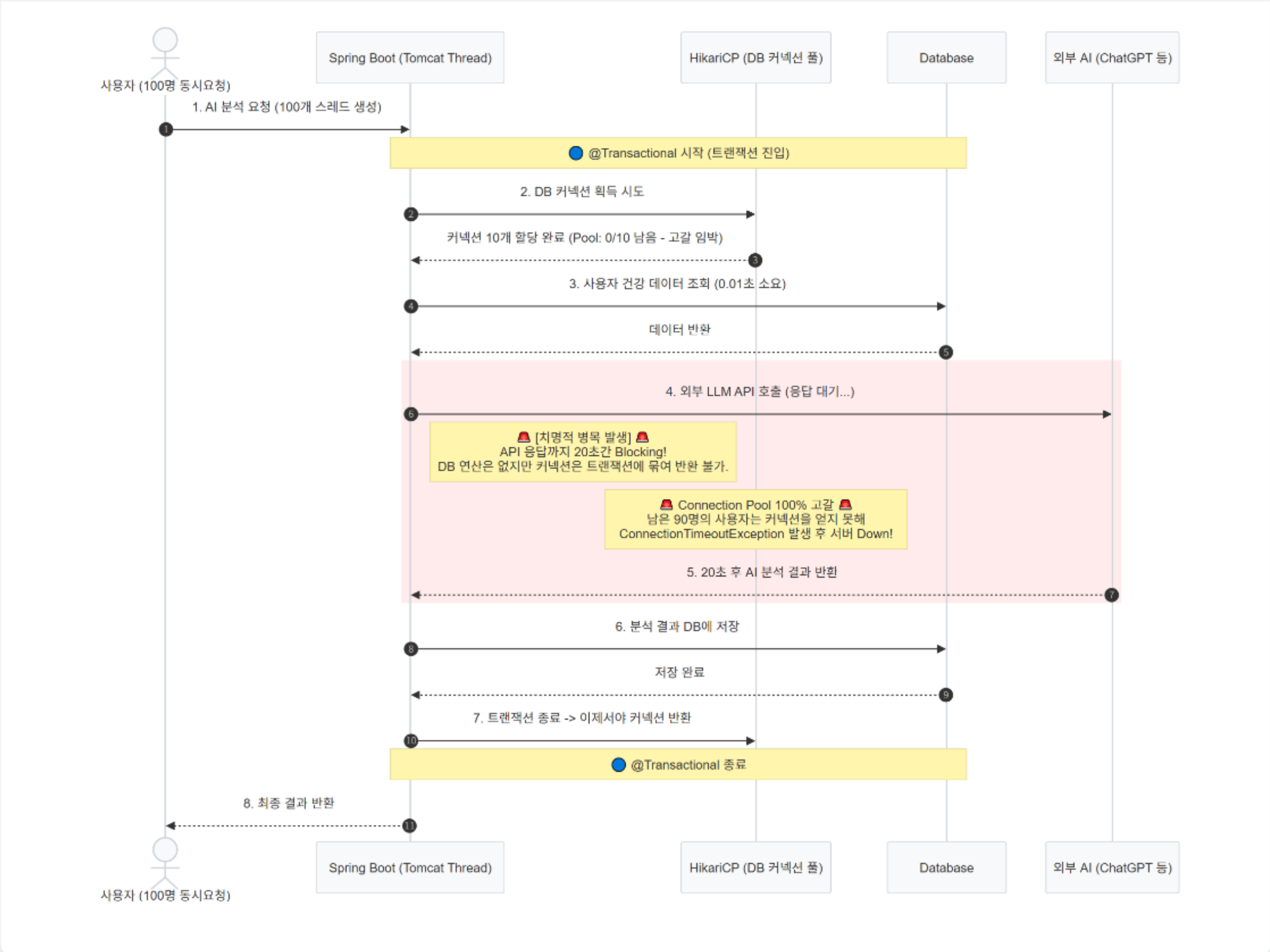


요구사항 분석 - Transaction

요구사항 분석

2 GB RAM 기준 권장
DB connection 수 10~20개

LLM API 호출 서비스는 반드시
트랜잭션에서 분리되어야 함



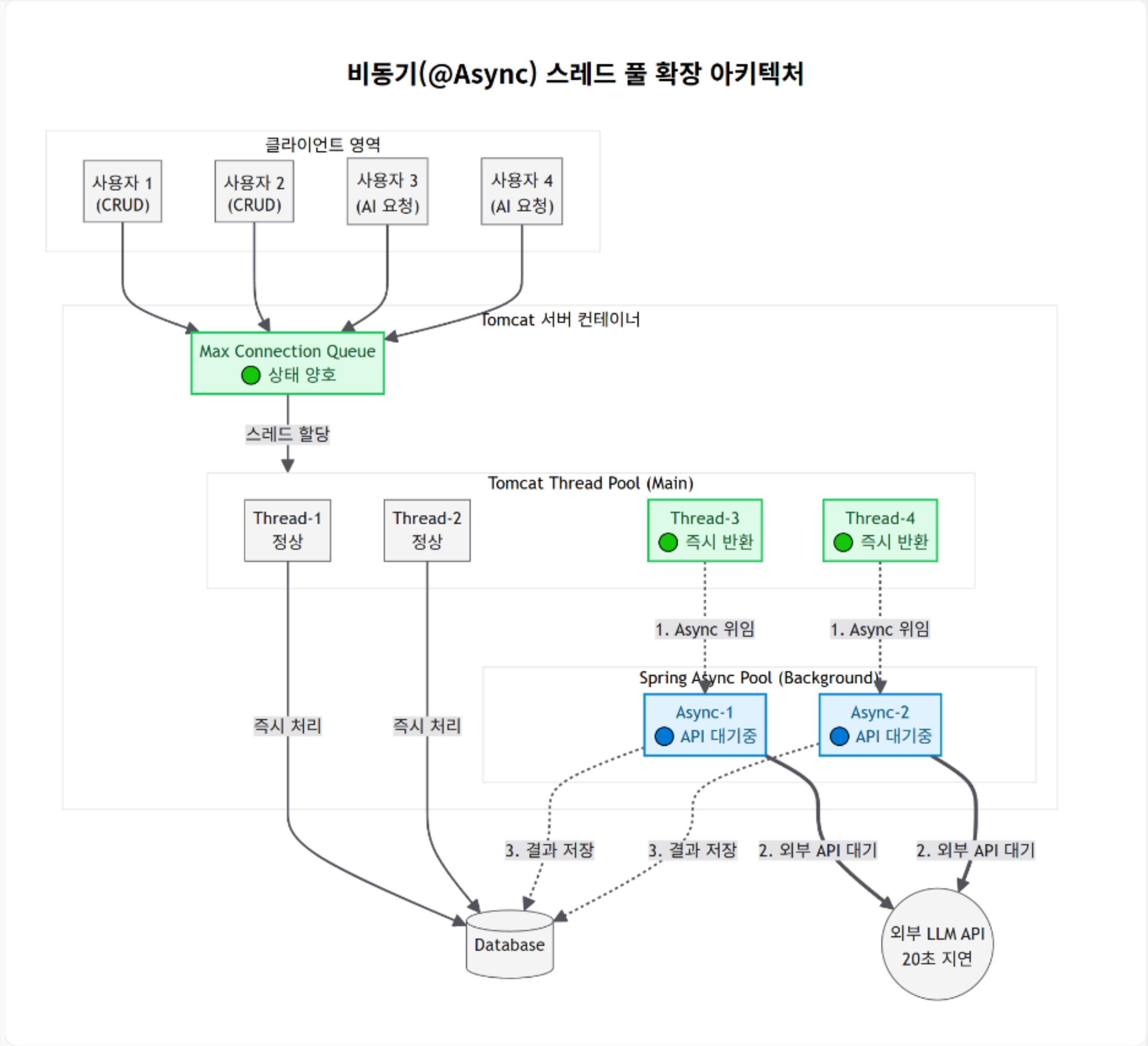
요구사항 분석 - MVP 선정

요구 사항 분석

최초 개발 목표인 동시 사용자 200명까지는 동기적으로 처리 가능.

빠른 출시 위해 동기적 처리로 선 출시 후 출시 기간동안 비동기 테스트 서버 개발.
동시 접속자 평균 150명 돌파 이후,
즉각 비동기 외부 API 처리 기능 도입.
수용가능 유저 500 % 향상 가능.

(물리 서버 확장 이전에 반드시 도입)



요구사항 분석 - MVP 선정

리틀의 법칙

$$L = \lambda \times W$$

L : 시스템 내에 유지되어야 하는 동시 작업 수 (필요한 Async 스레드 개수)

λ : 목표 처리량 (초당 들어오는 LLM 분석 요청 수, TPS)

W : 작업 하나당 걸리는 시간 (LLM 응답 대기 시간, 약 20초)

목표 초당 5건 처리 $5 \times 20 = 100$

100개의 Async 스레드 필요

요구사항 분석 - MVP 선정

요구 사항 분석

평균 스레드 점유 지표 확인. Spring actuator 사용.

주요 수집 지표(Metric)

tomcat.threads.busy : 현재 실제로 요청을 처리 중인 스레드 수

tomcat.threads.current : 현재 생성되어 있는 전체 스레드 수

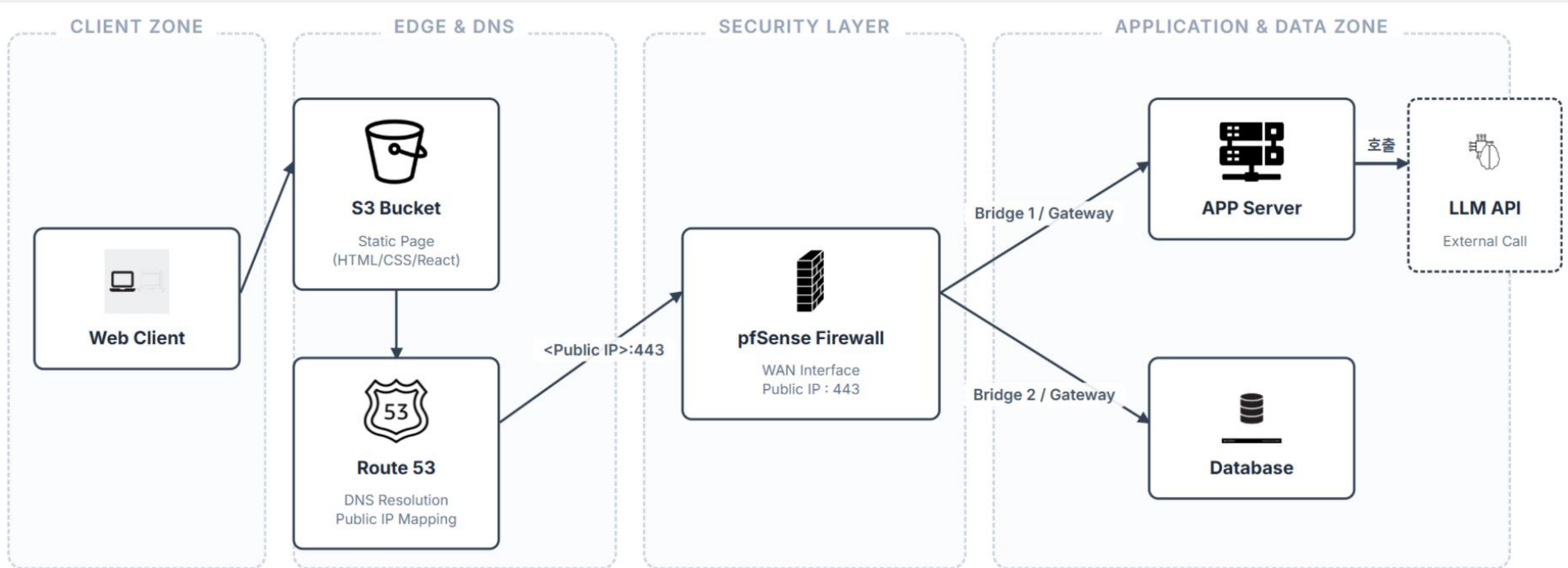
tomcat.threads.config.max : 설정된 최대 스레드 수

tomcat.threads.config.min : 최소로 유지하는 유휴 스레드 수



**Spring Boot
Actuators**

시스템 아키텍처

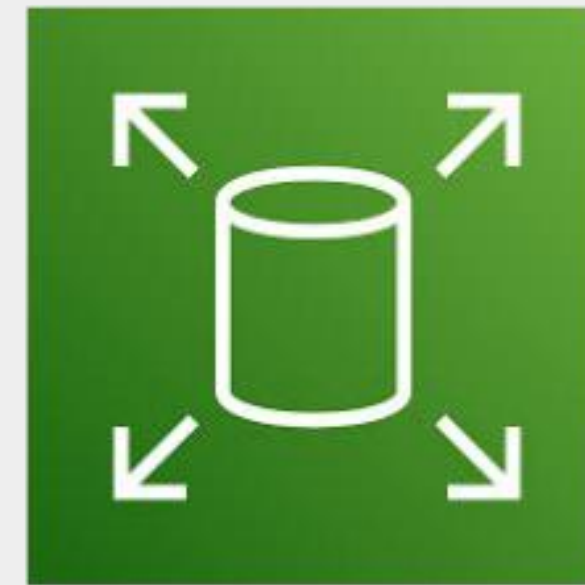


Cost (if AWS MVP)

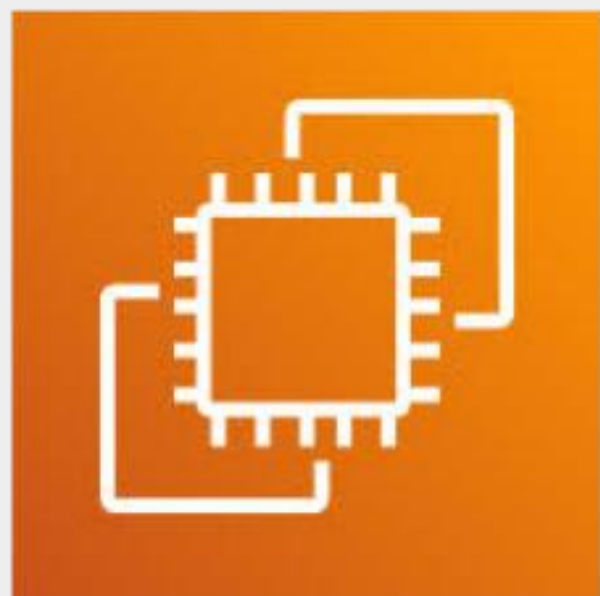
한 달 기준



S3 Hosting : \$ 0.005



EBS : 30GB : \$2.40



Server : t3.small: \$18.10

Eip : \$3.65

Nat EC2 : t3.nano : \$3.80

(의료데이터 서비스 서버 공인망 x)



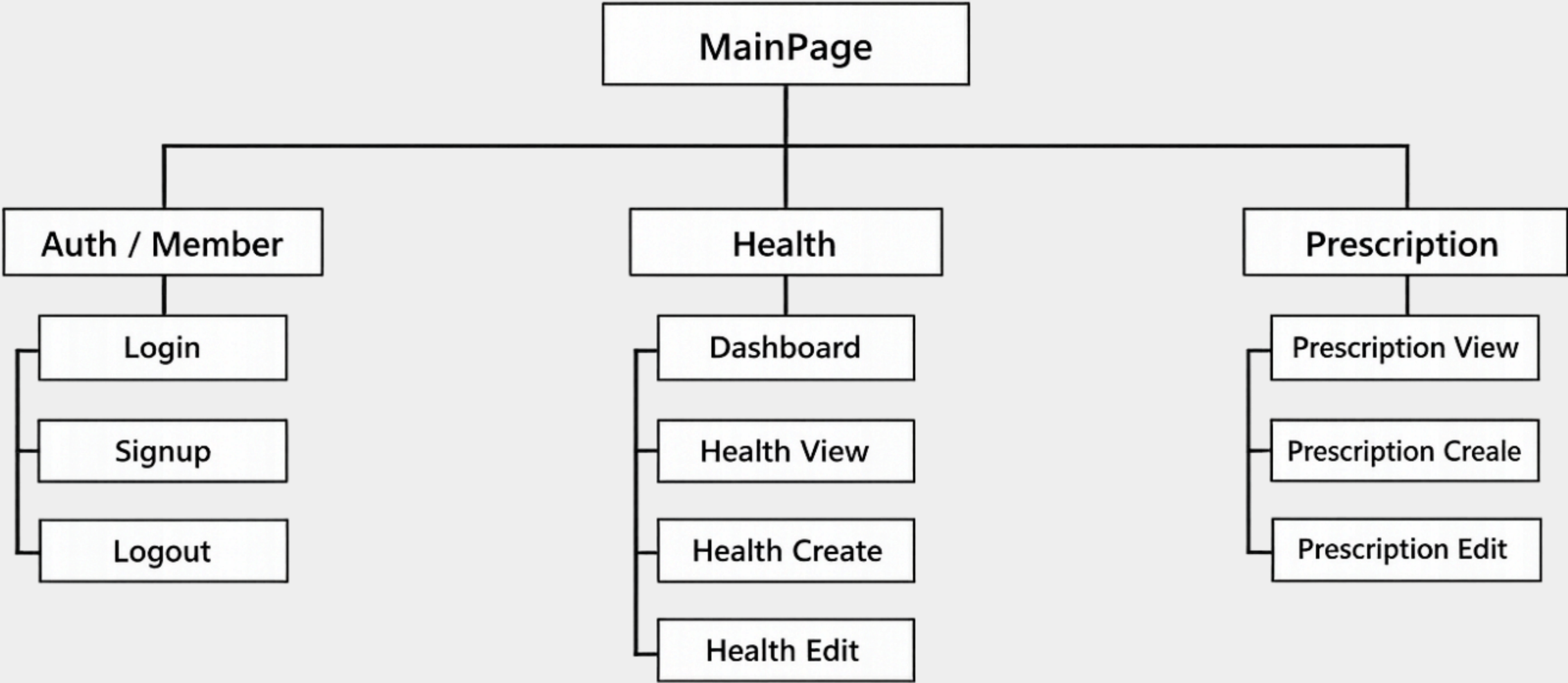
RDS

db.t4g.micro : \$14

총합 : 약 \$42

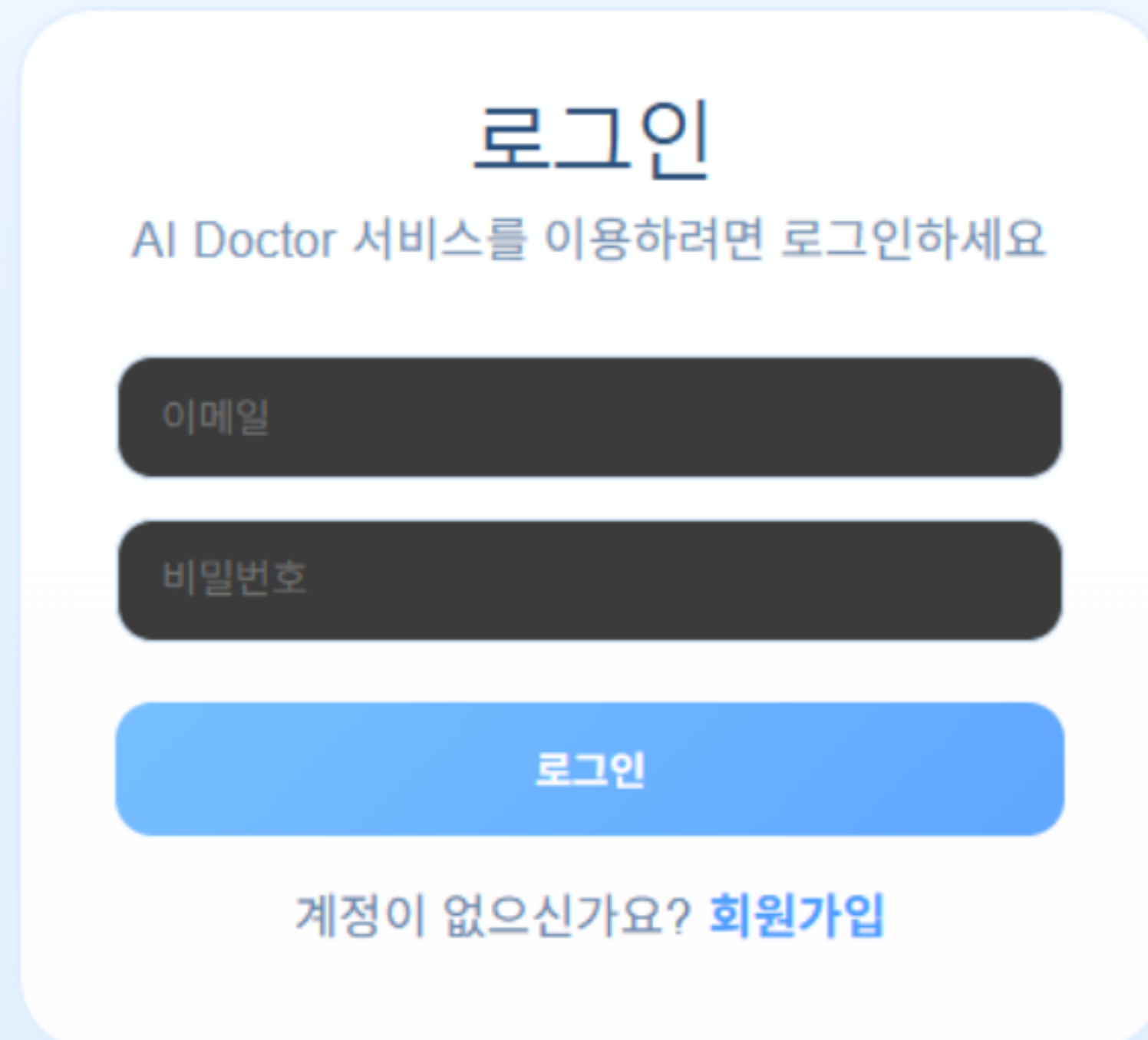
UI/UX

전체 UI/UX 구조 소개
인증-회원 도메인
건강 도메인
처방 도메인
전체 매핑 정리



UI/UX

LoginPage - 로그인 화면



The login form is centered on a light blue background. It features a white rounded rectangle containing the title '로그인' in bold blue text. Below the title is a subtitle 'AI Doctor 서비스를 이용하려면 로그인하세요' in a smaller blue font. There are two dark gray input fields: the first is labeled '이메일' and the second is labeled '비밀번호'. Below these fields is a blue button with the text '로그인'. At the bottom of the form, there is a link '계정이 없으신가요? 회원가입' in blue text.

- 아이디/비밀번호 기반 인증 화면
- AccessToken + RefreshToken Cookie 발급
- 반복 실패 시 오류 메시지를 표시하고 재입력을 유도한다

성공 시 다시 보드로 이동

API 연동

POST /api/member/login

POST /api/member/token/refresh

POST /api/member/logout

데이터 타입

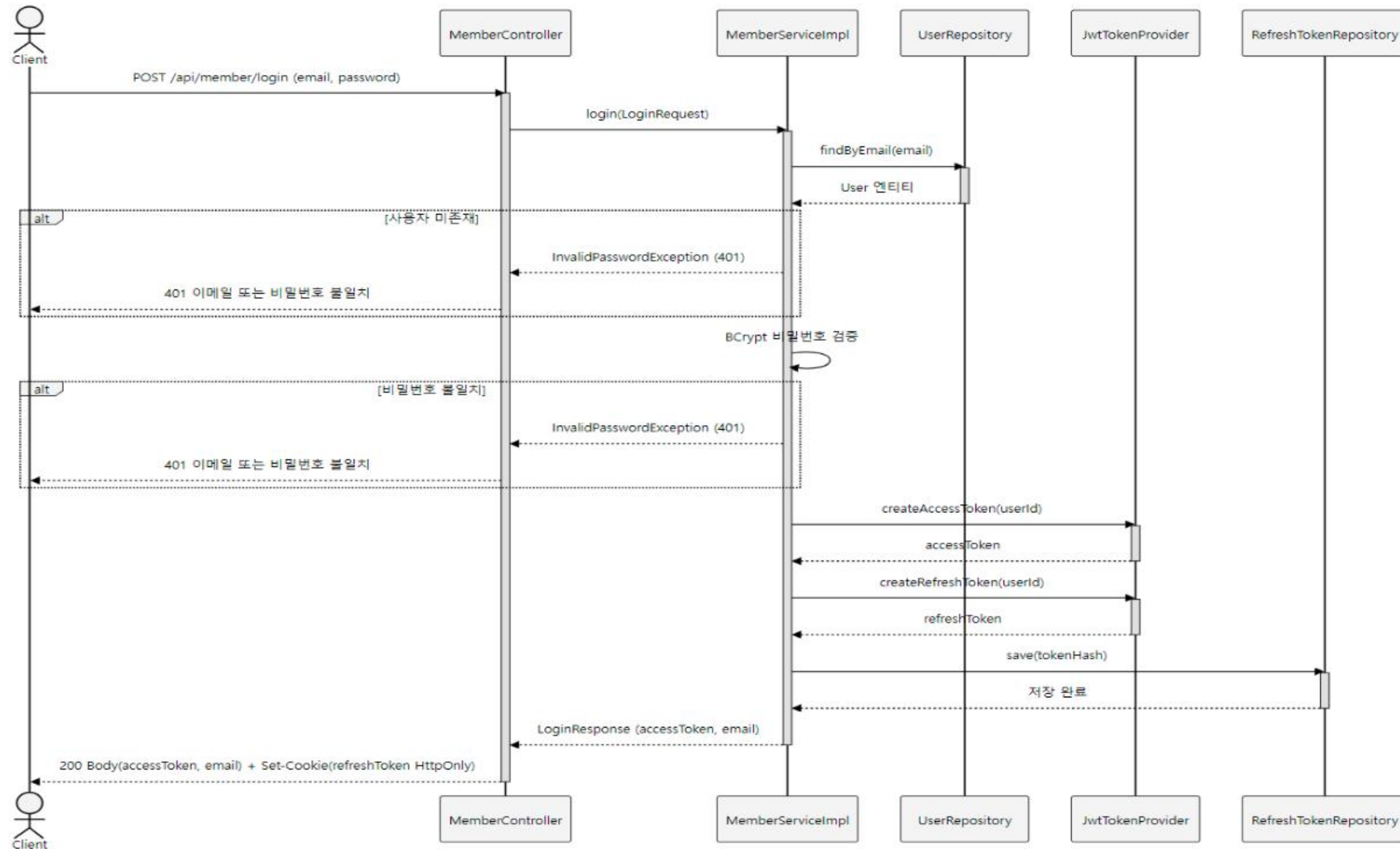
Request
email: String
password : String

Response
status : int
message : String
data.accessToken : String
Set-Cookie : RefreshToken

Auth / Members Sequence Diagram

POST

/api/member/login



UI/UX

회원 가입 화면

회원가입

AI Doctor 서비스를 시작하려면 회원정보를 입력하세요

이름

이메일

비밀번호

회원가입

이미 계정이 있으신가요? [로그인](#)

- 아이디.비밀번호.이름.이메일.생년월일.성별 입력 후 계정 생성

.

- 전화번호는 선택 입력 항목으로 처리
- 이메일 중복 시 서버 즉시 검증

성공 시로 이동
API연동 POST `/api/member/signup`

데이터 타입

Request

name : String
email : String
password : String

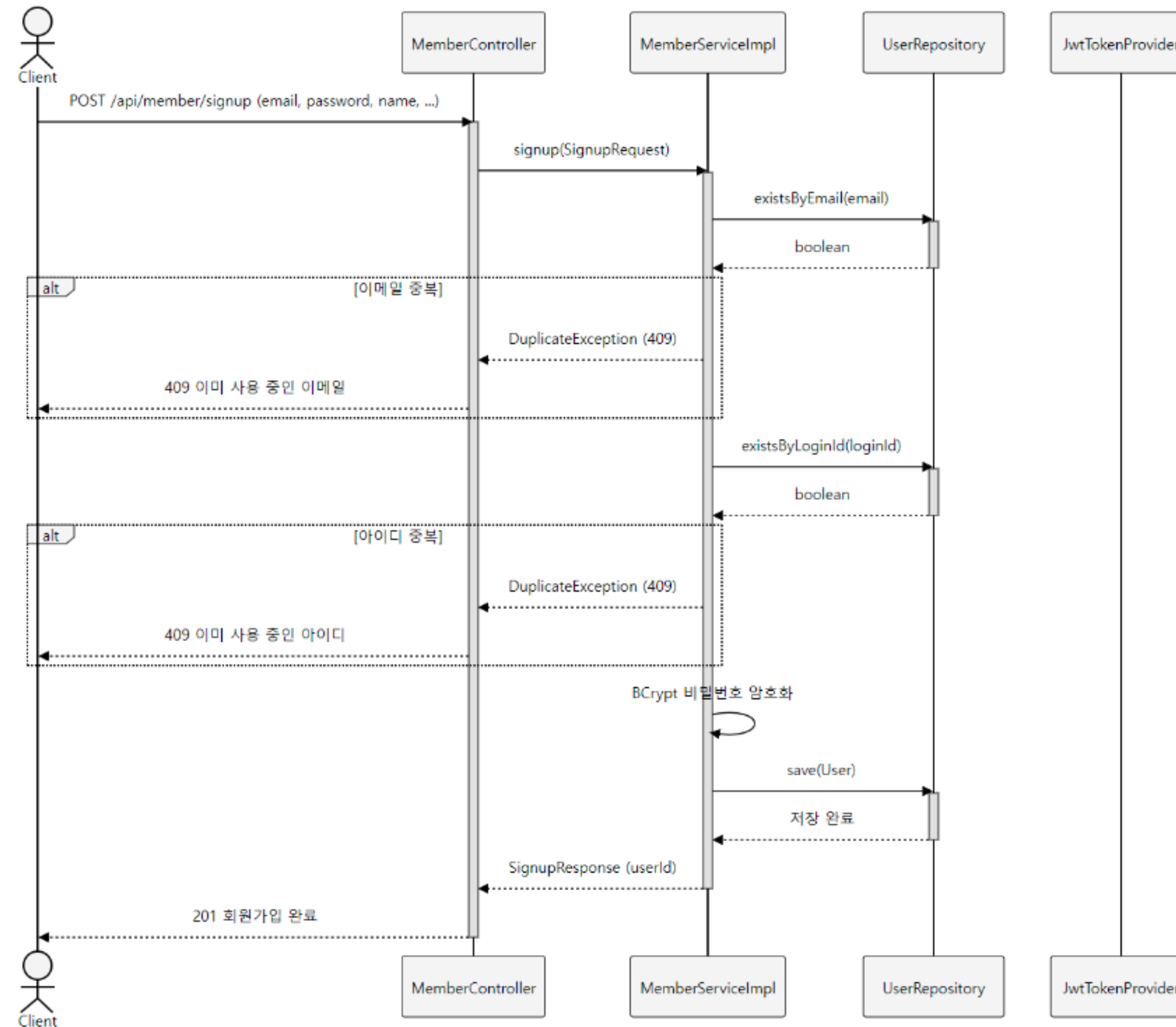
Response

status : int
message : String

Auth / Members Sequence Diagram

POST

/api/member/signup



UI/UX

Auth / Member UI - 로그인 상태 기반 Navbar



- 상단 Navbar는 로그인 상태에 따라 다른 메뉴를 제공한다.
- 비로그인 상태에서는 로그인/회원가입 버튼을 표시하고, 로그인 상태에서는 대시보드/로그아웃 버튼을 표시한다.
- [로그아웃] 클릭 → RefreshToken 폐기 후 메인 화면으로 이동

API 연동

POST /api/member/logout

데이터 타입

Request Header

Authorization : Bearer
{Token}

Response

status : int / message : String

->RefreshToken 쿠키 삭제

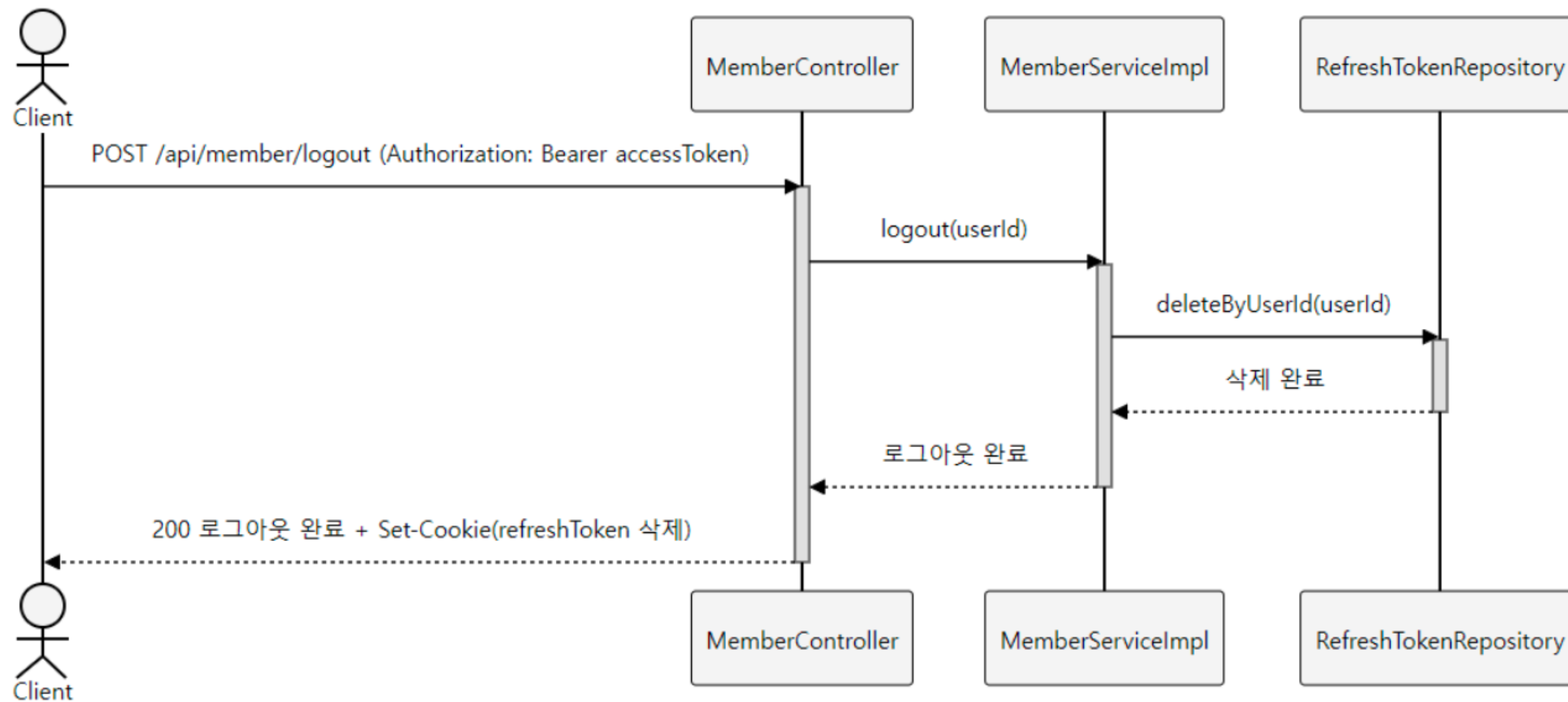
->localStorage accessToken 제거

->navigate('/') 메인 이동

Auth / Members Sequence Diagram

POST

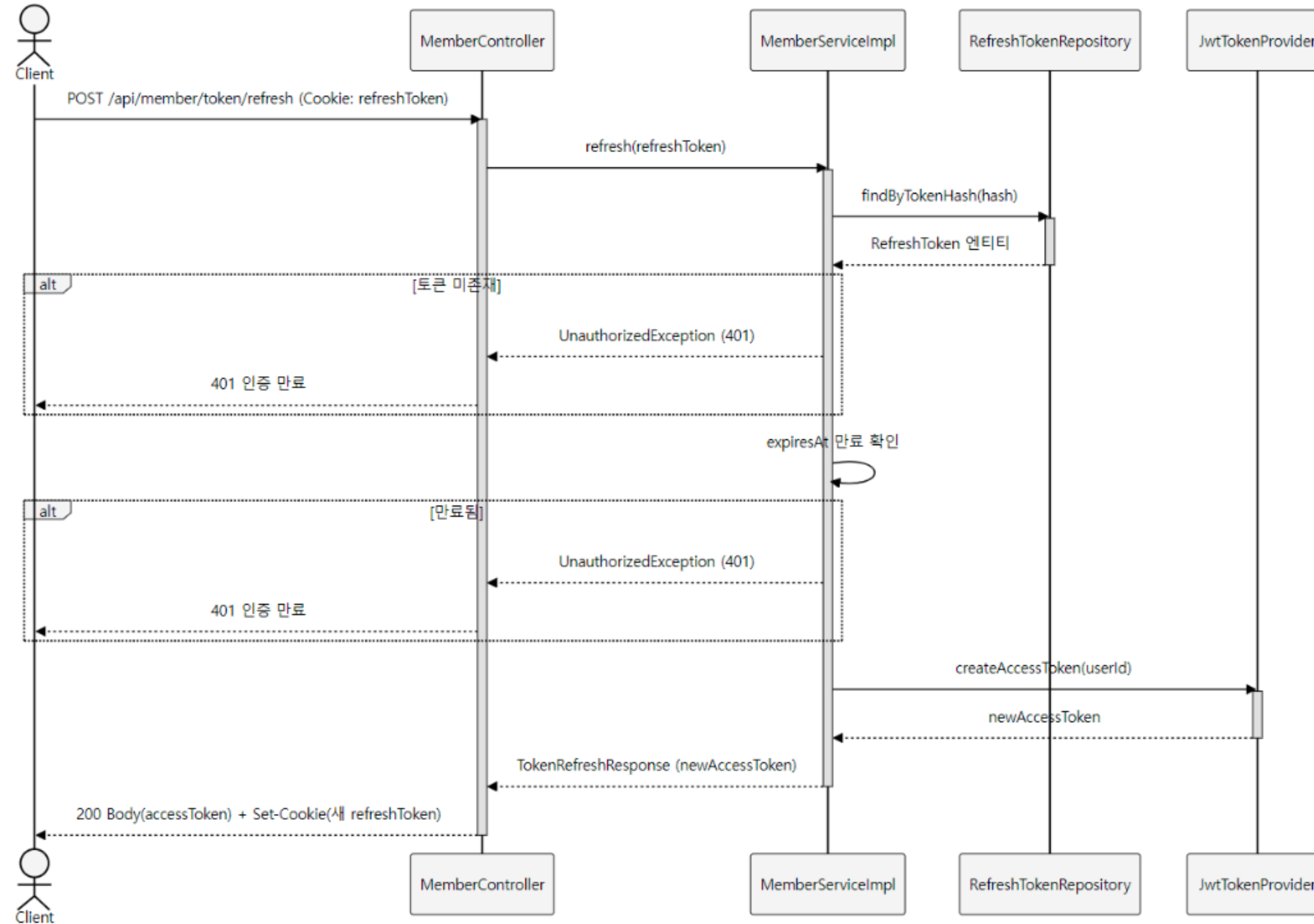
/api/member/logout



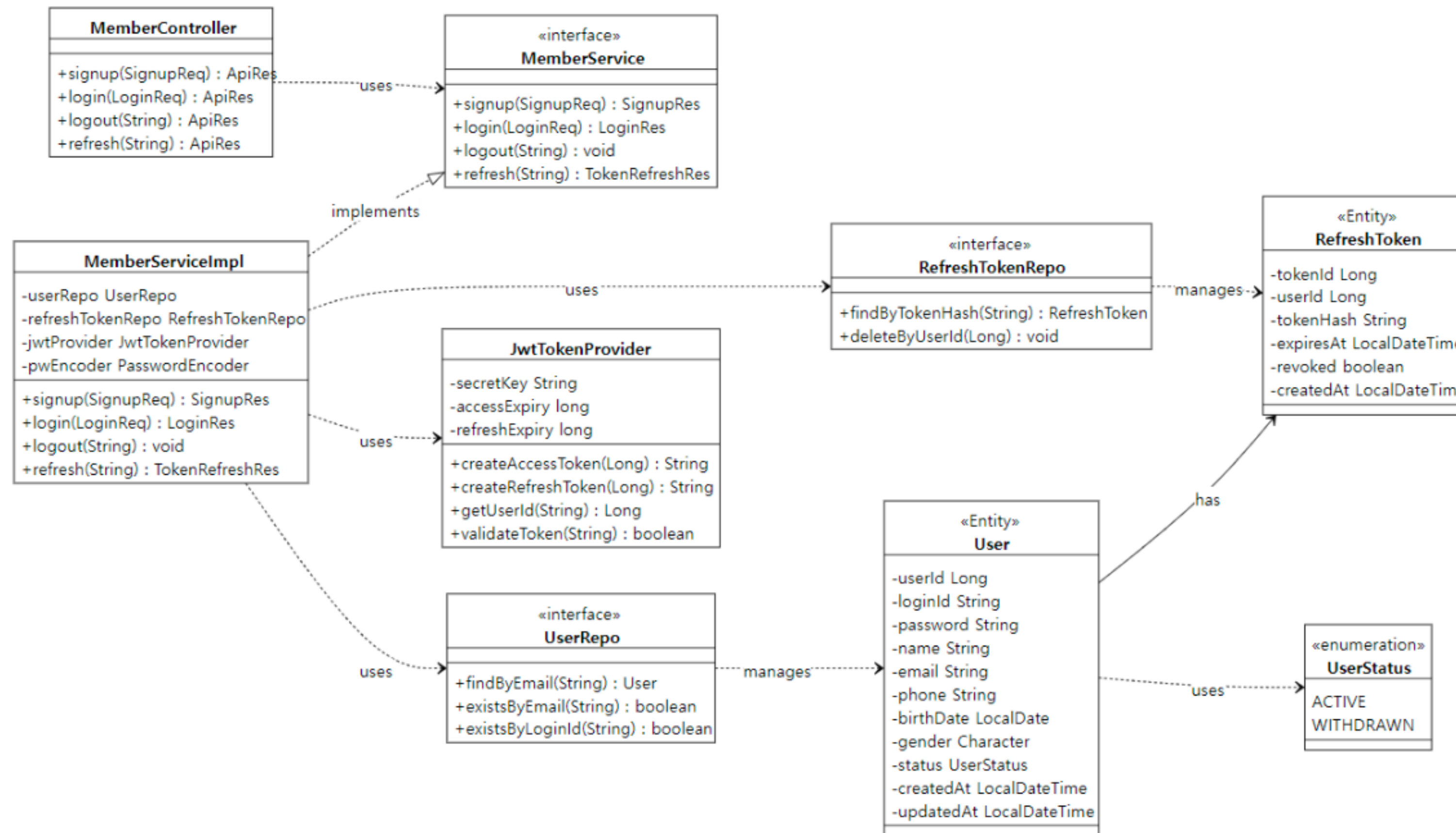
Auth / Members Sequence Diagram

POST

/api/member/token/refresh

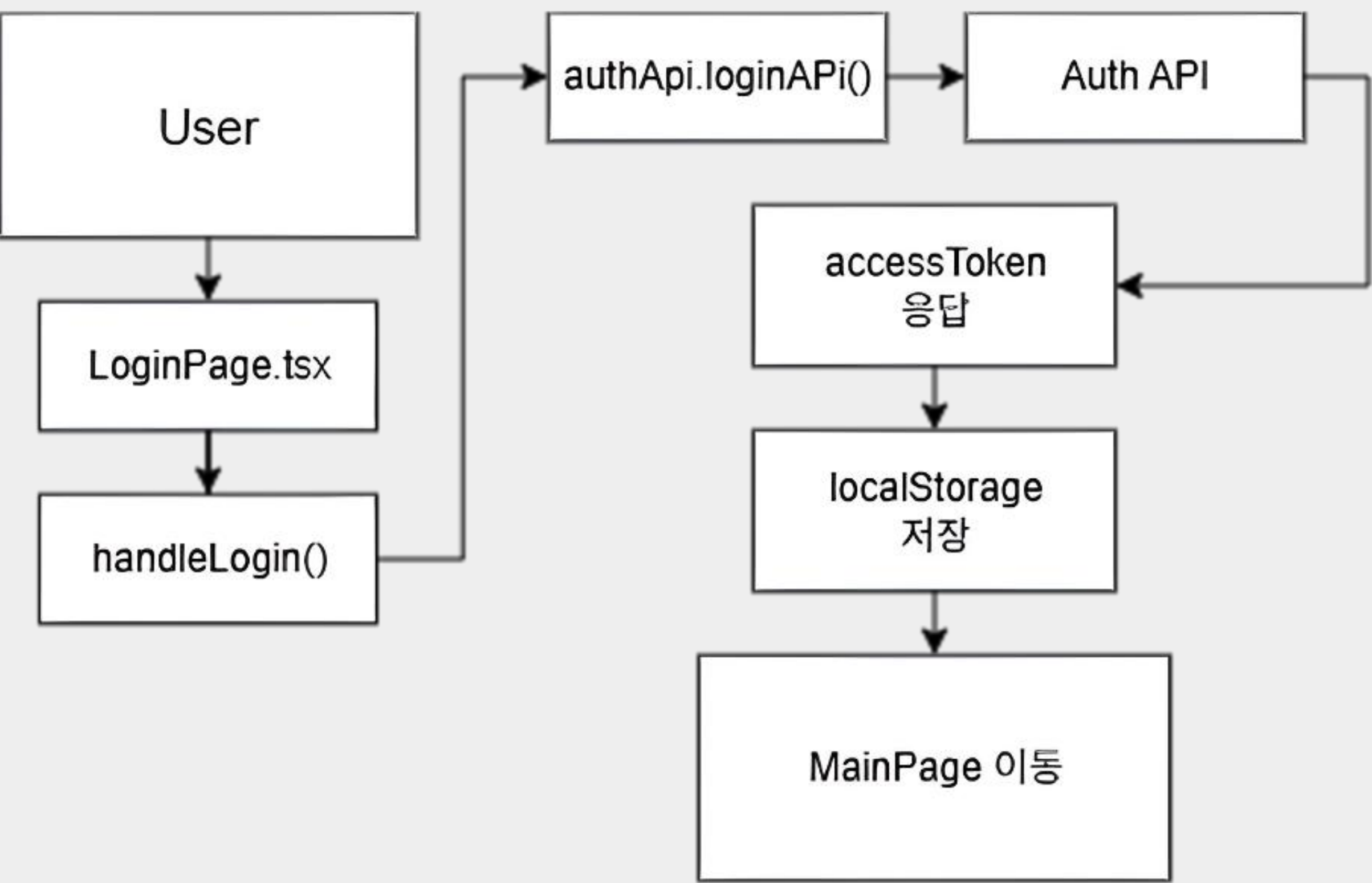


Members Domain Class Diagram



Auth / Members API Flow

API Flow Sequence



- 사용자가 이메일·비밀번호 입력 후 LoginPage에서 입력값 검증
- handleLogin() 호출 → authApi.loginApi() 로 Auth API 요청 전송
- 응답받은 AccessToken과 이메일을 localStorage에 저장
- 저장 완료 후 MainPage로 이동

API 연동

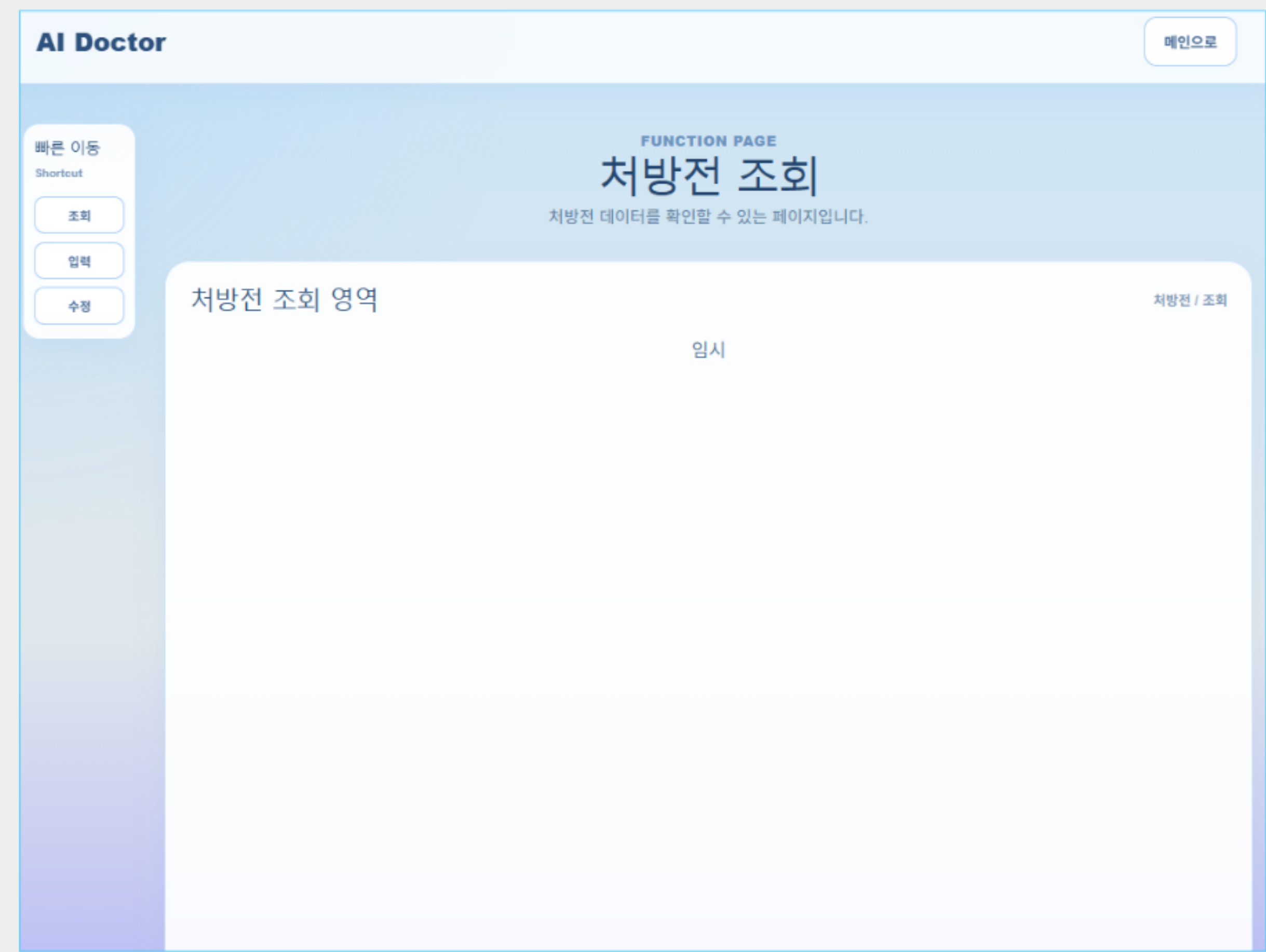
POST /api/member/login

데이터 타입

Request	Response
email : String	accessToken : String
password : String	email : String (localStorage 저장)

UI/UX

처방전 조회 화면



- 등록된 처방전 목록 카드형 조회
- 수정 버튼 -> 처방전 수정 이동
- 삭제 버튼 -> 확인 다이얼로그 후 삭제 상태로 변경
- 데이터 없을 시 안내 메시지 표시

API 연동

POST /api/prescription/list

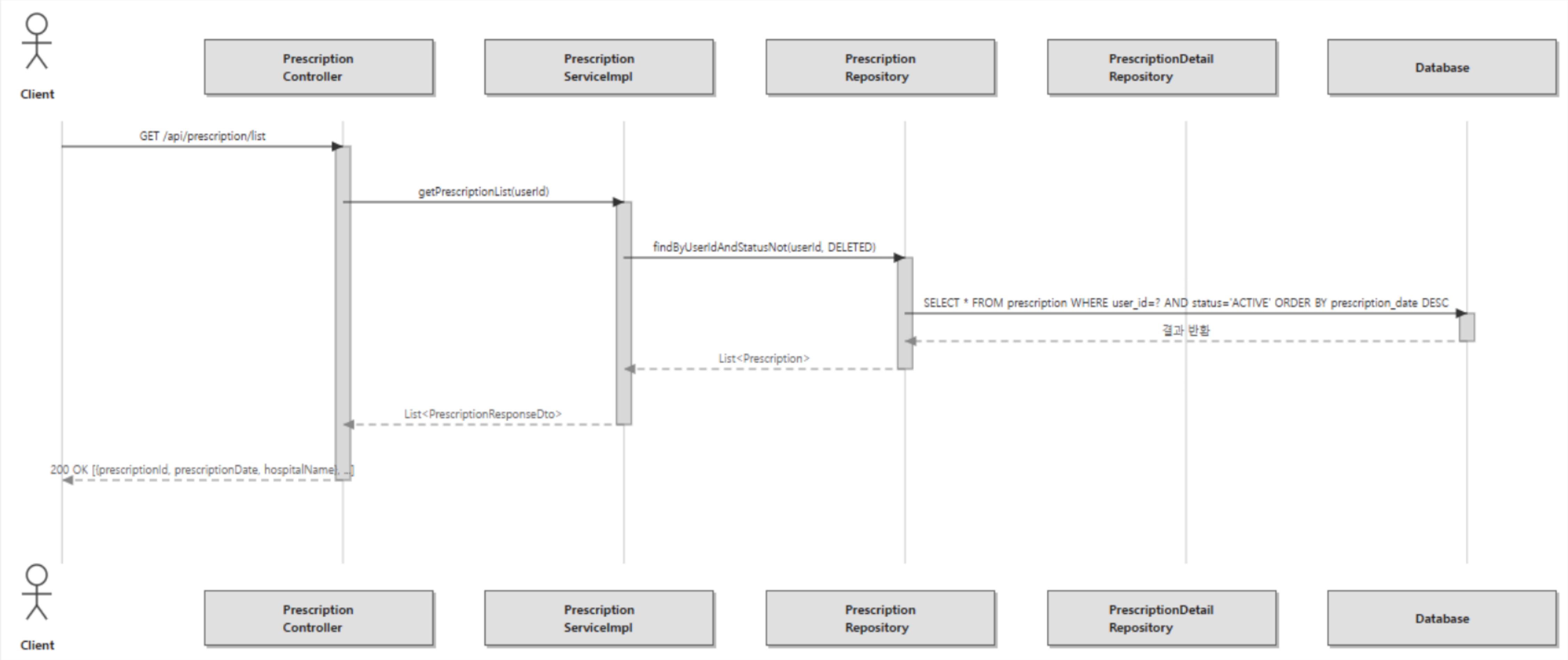
DELETE /api/prescription/delete/{prescription_id}

데이터 타입

prescriptionId : Long
prescriptionDate : String
hospitalName : String
medicineList : Array

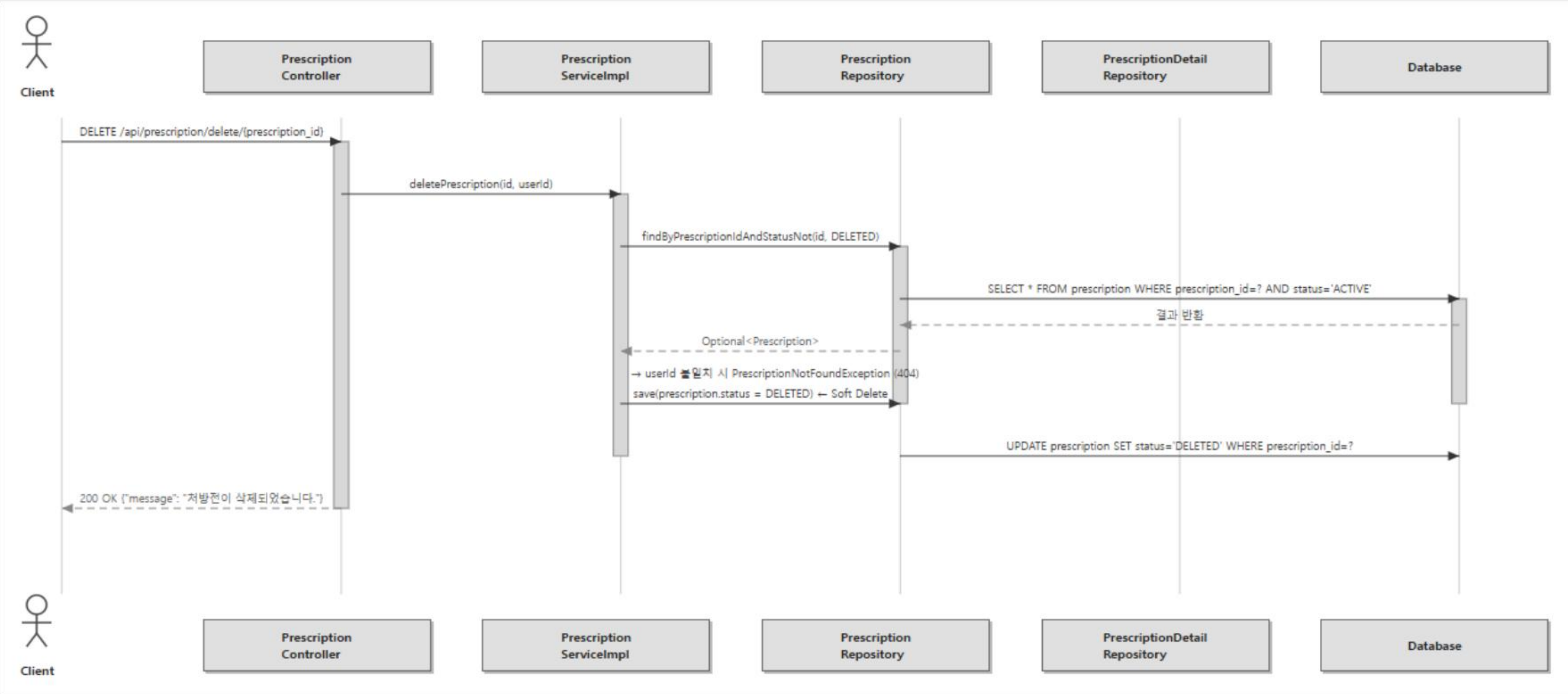
Prescription Sequence Diagram

GET /api/prescription/list



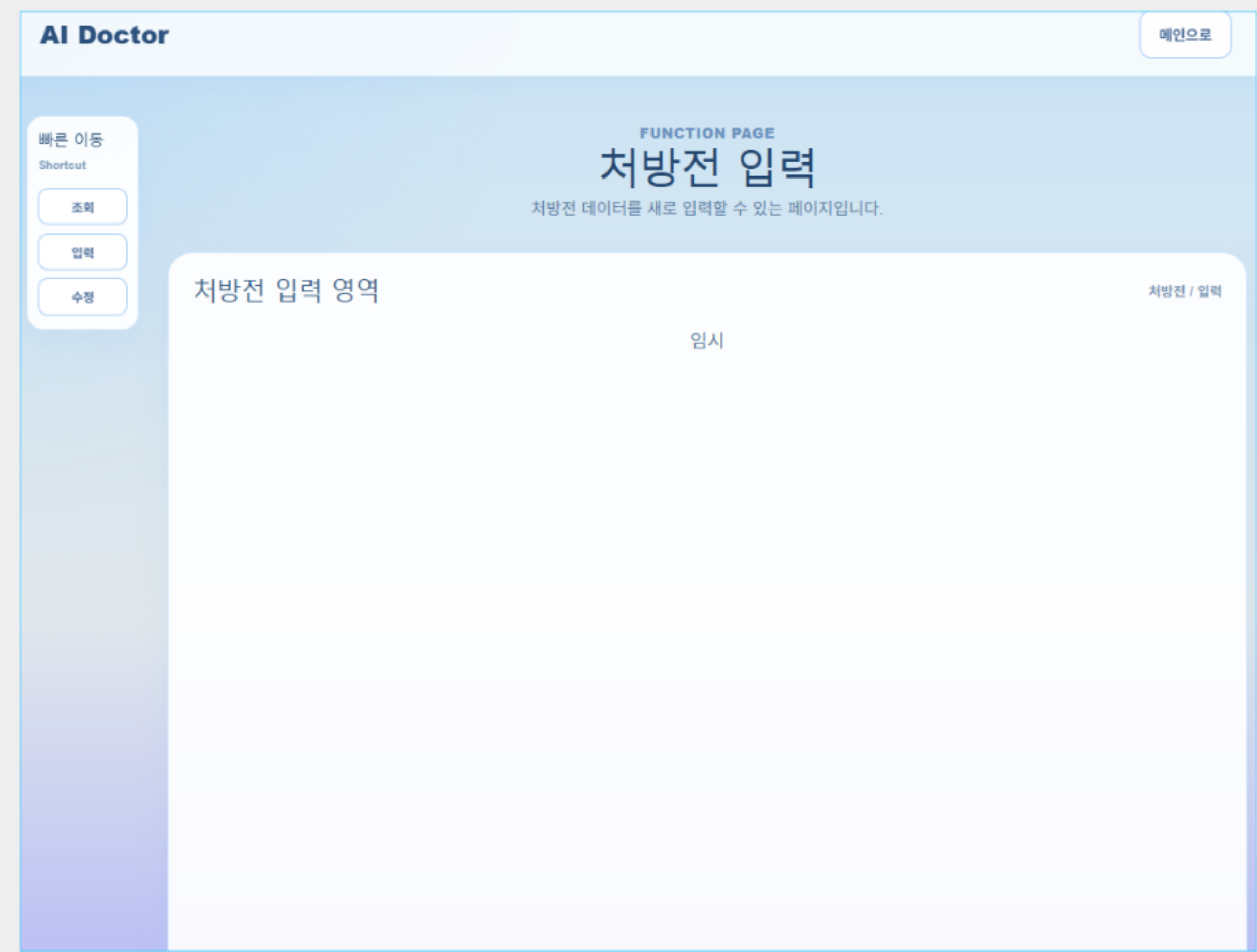
Prescription Sequence Diagram

DELETE /api/prescription/delete/{prescription_id}



UI/UX

처방전 입력 화면



- 처방전 + 처방약 정보 등록 화면
- 처방일 : 미래 날짜 선택 불가
- 필수 항목 미입력 시 저장 차단
- 저장 성공 -> 처방전 조회로 이동

API 연동

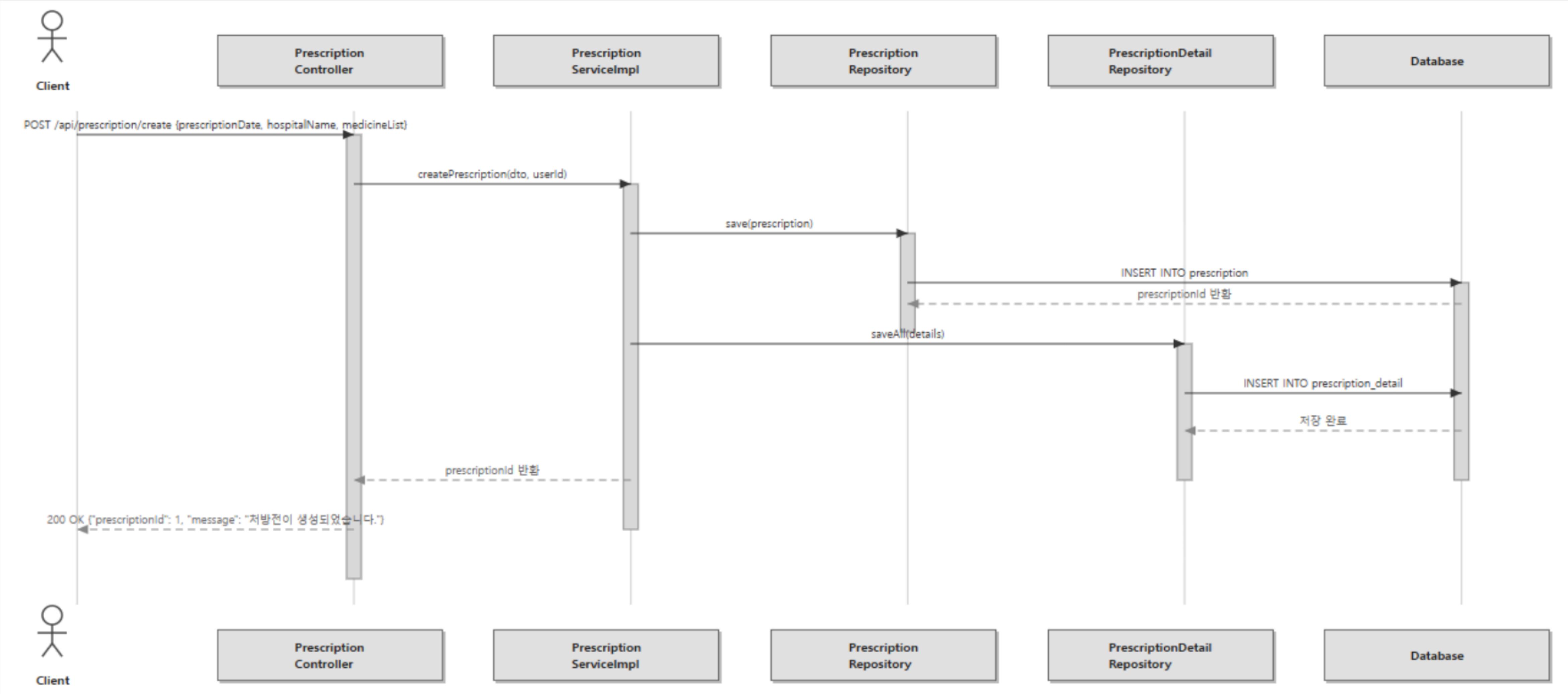
POST /api/prescription/create

데이터 타입

prescriptionDate : String
hospitalName : String
medicineList[].medicineName : String
medicineList[].dosage : String
medicineList[].duration : String
-> data.prescriptionId : Long

Prescription Sequence Diagram

POST /api/prescription/create



UI/UX

처방전 수정 화면



- 화면 진입 시 기존 처방전 데이터를 자동 로드한다
- 수정 완료 시 PUT 요청으로 기본 정보와 처방약 목록을 갱신한다
- 수정 성공 시 처방전 조회 화면으로 이동한다

API 연동

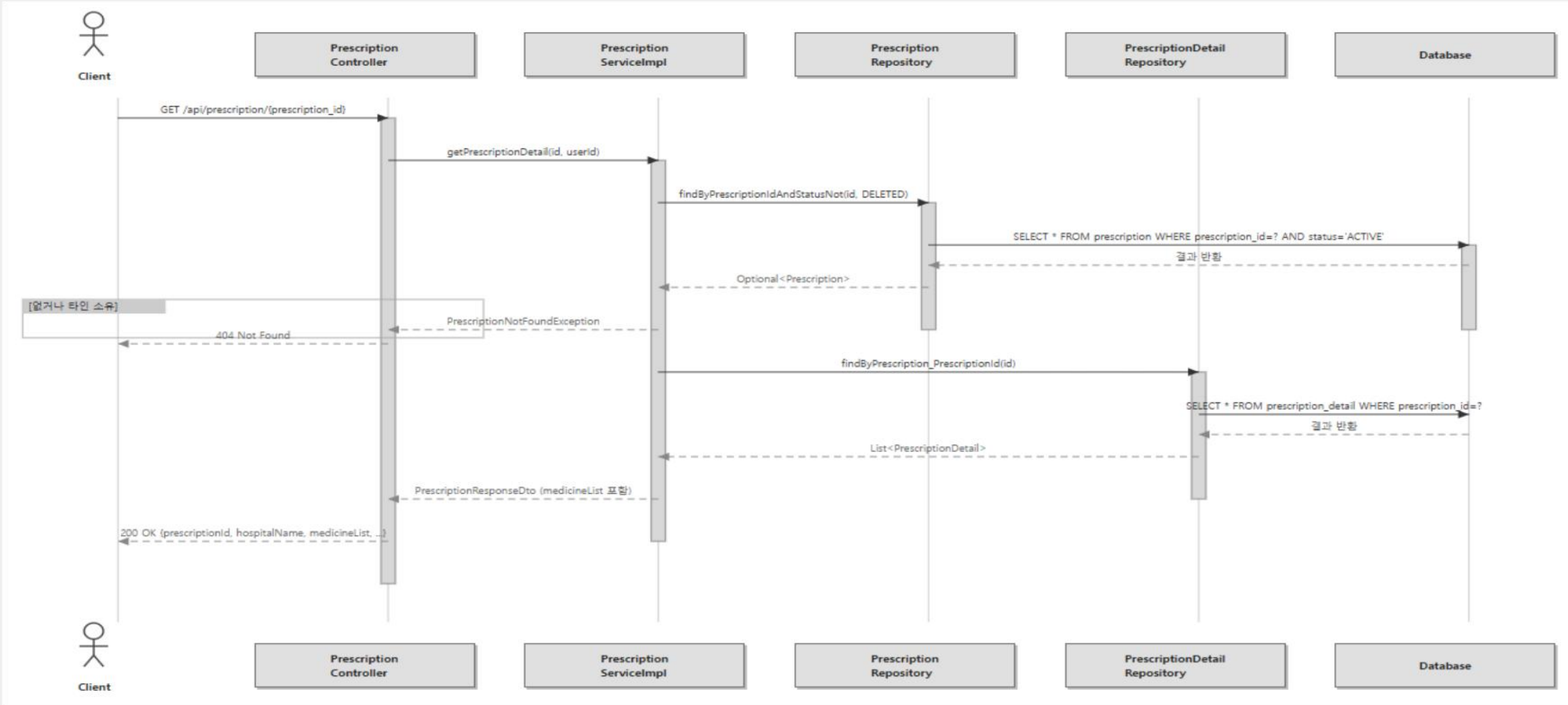
GET	/api/prescription/{prescription_id}
PUT	/api/prescription/update/{prescription_id}

데이터 타입

```
prescription_id : Long  
prescriptionDate : String  
hospitalName : String  
medicineList : Array  
-> message : String
```

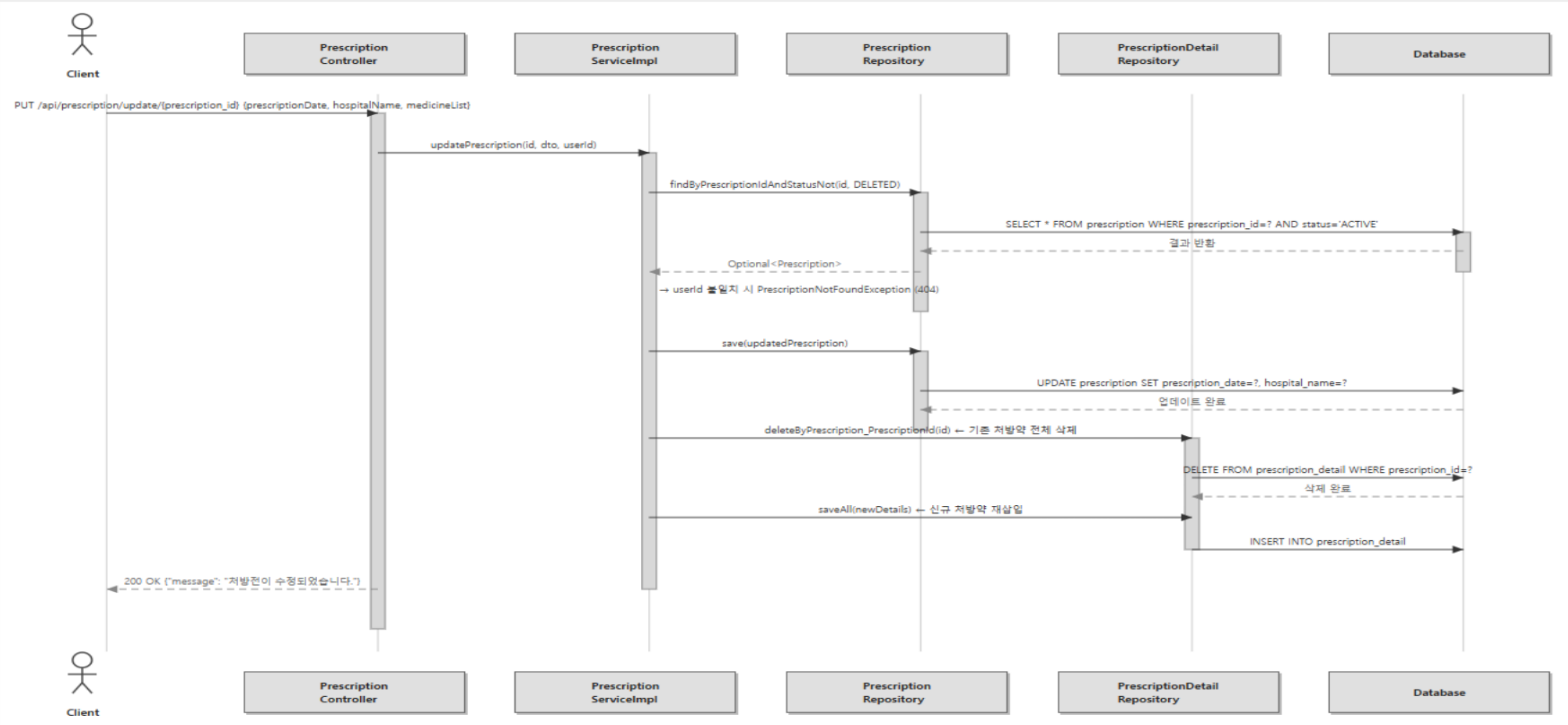

Prescription Sequence Diagram

GET /api/prescription/{prescription_id}

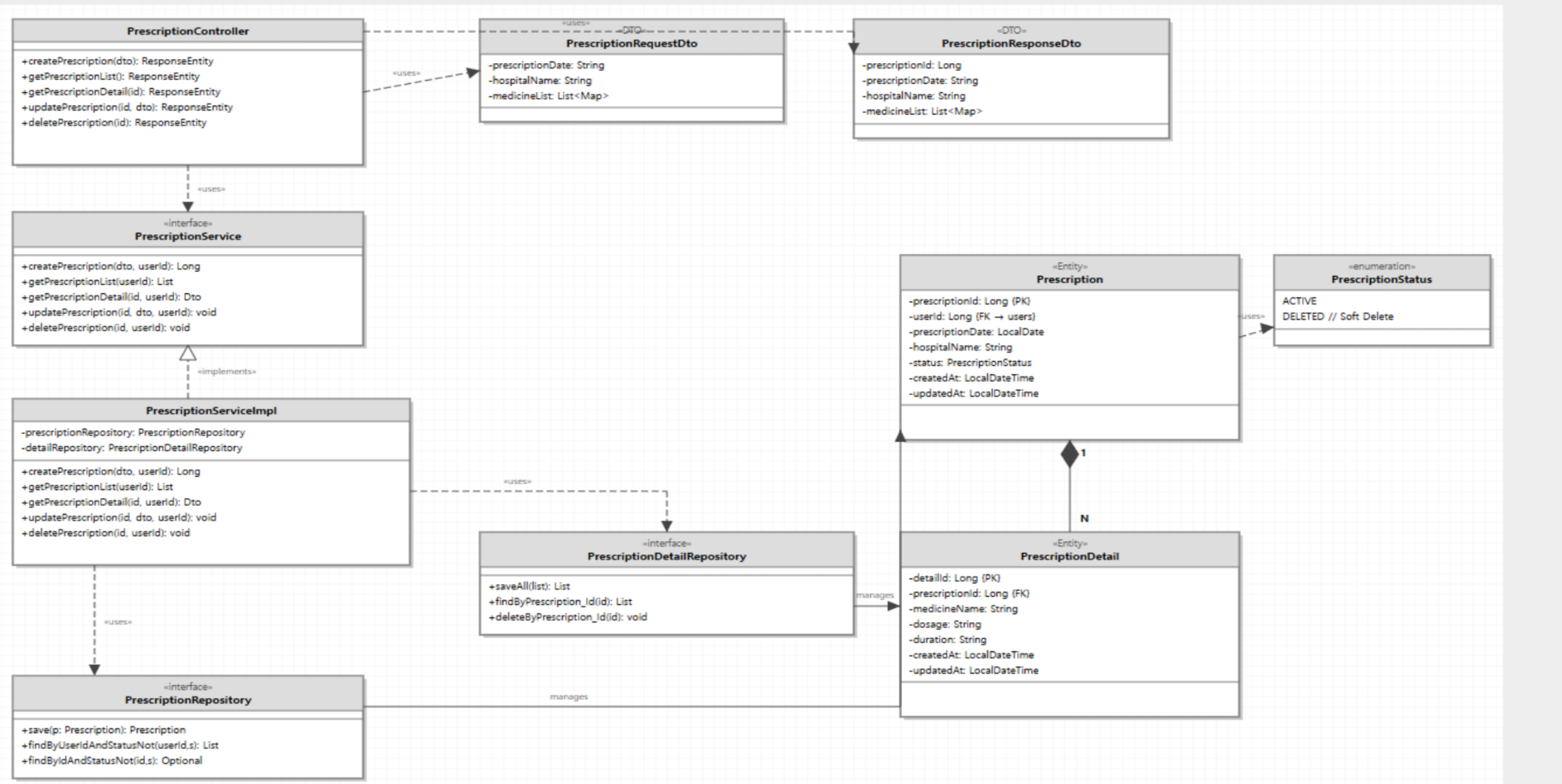


Prescription Sequence Diagram

PUT /api/prescription/update/{prescription_id}



Prescription Domain Class Diagram



UI/UX

건강정보 조회 화면



- 건강 이력을 날짜 기준 목록 조회
 - 수정 버튼 -> 건강정보 수정 이동
 - 삭제 버튼 -> 확인 다이얼로그 후 삭제 상태로 변경
- ※ API 경로는 백엔드 최종 명세 기준으로 확정 예정

API 연동

GET /api/health/list

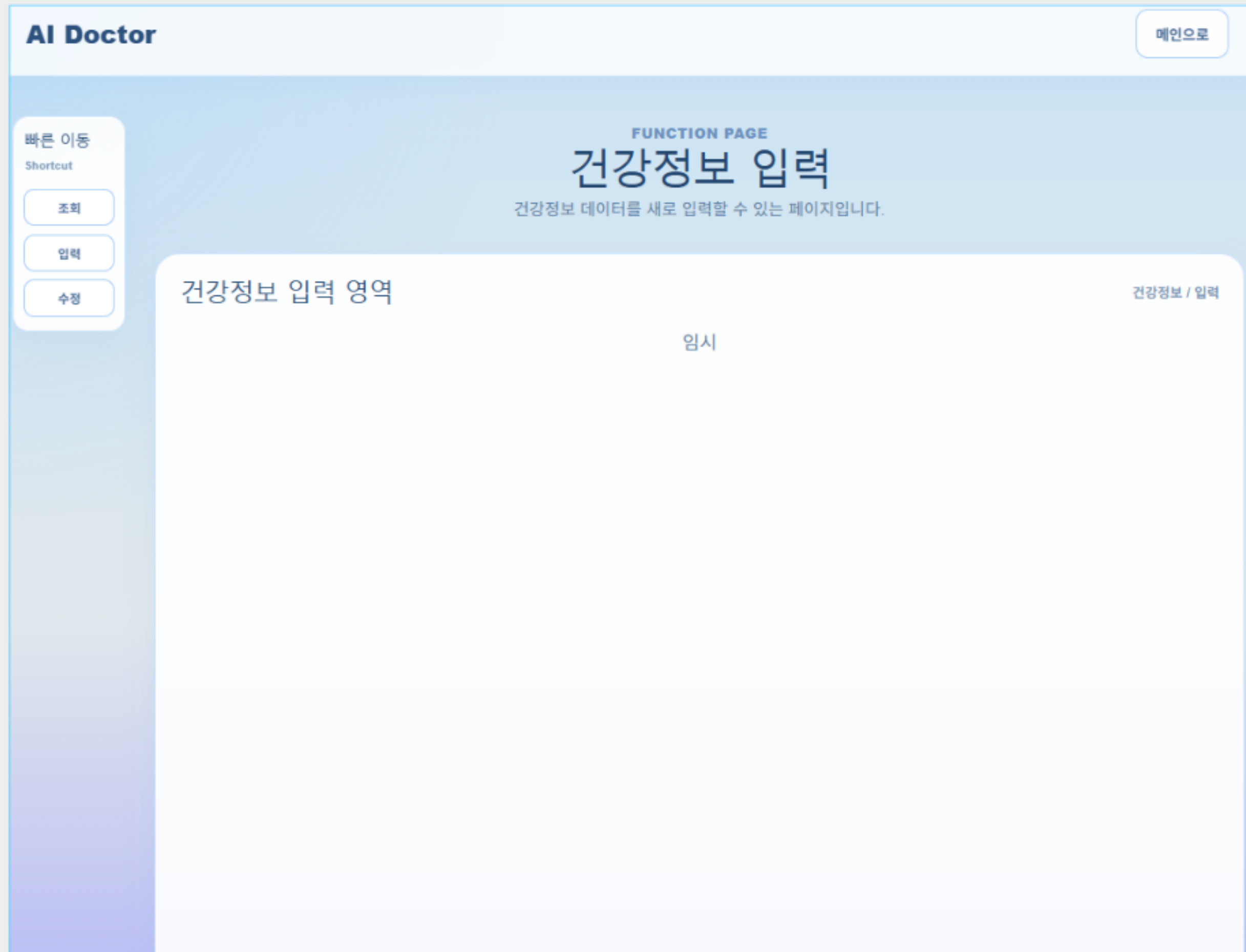
DELETE /api/health/{health_id}

데이터 타입

healthId : Long
recordDate : String
symptom : String
history : String
note : String

UI/UX

건강정보 입력 화면



- 선택형 토글 방식으로 증상 항목 입력
- 필수 항목 미입력 시 저장 요청 차단
- 입력값은 healthApi를 통해 서버에 전달
- 저장 성공 시 건강정보 조회 화면으로 이동

API 연동

POST /api/health

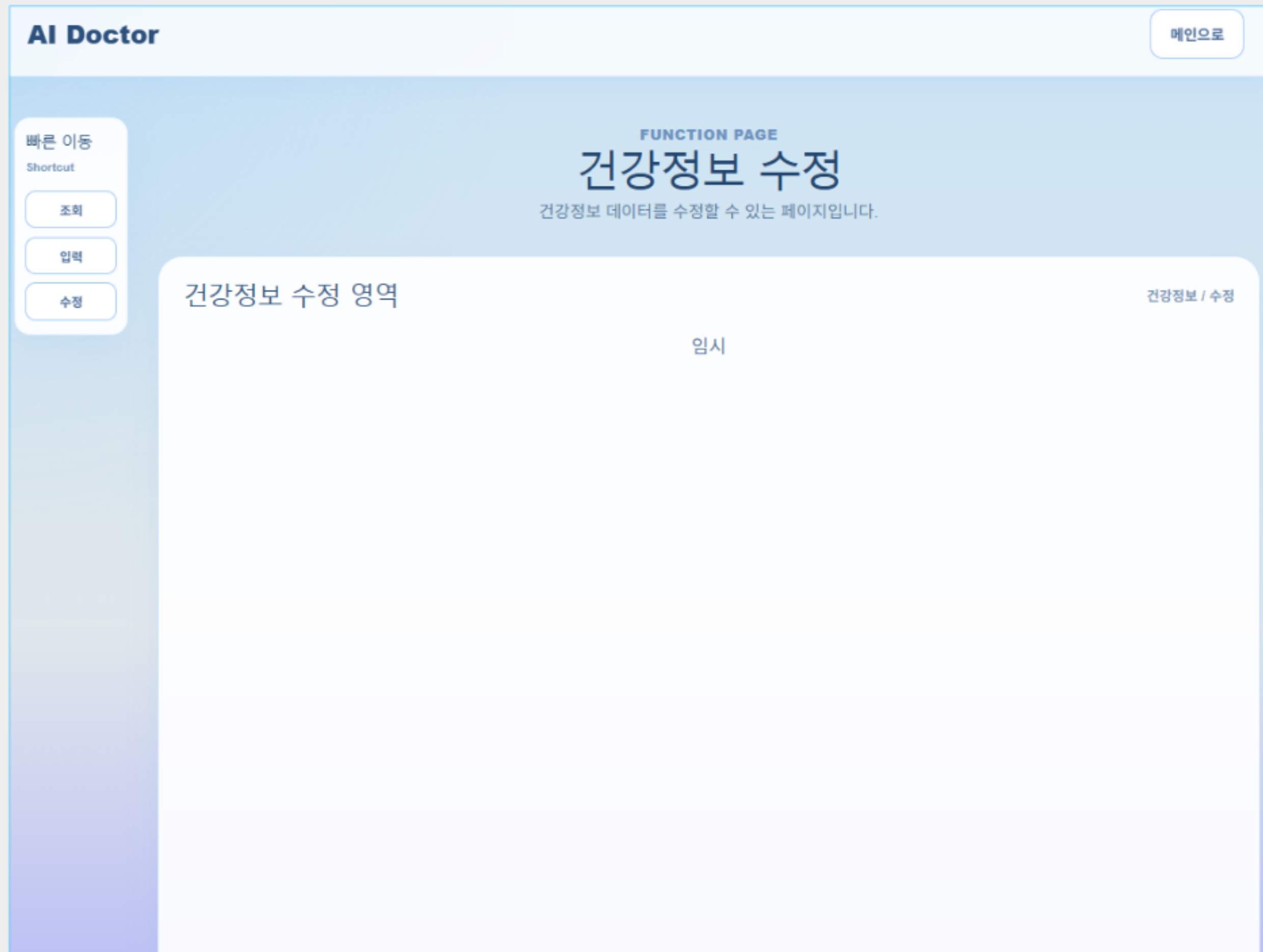
데이터 타입

Request
recordDate : String
symptom : String
history : String
note : String

Response
status : int
message : String

UI/UX

건강정보 수정 화면



- 진입 시 기존 건강 데이터 자동 로드
- 기존 증상 토글 상태도 자동 반영
- 수정 완료 -> 건강정보 조회 이동
- 이상값 범위 초과 시 경고 팝업

API 연동

GET /api/health/{health_id}

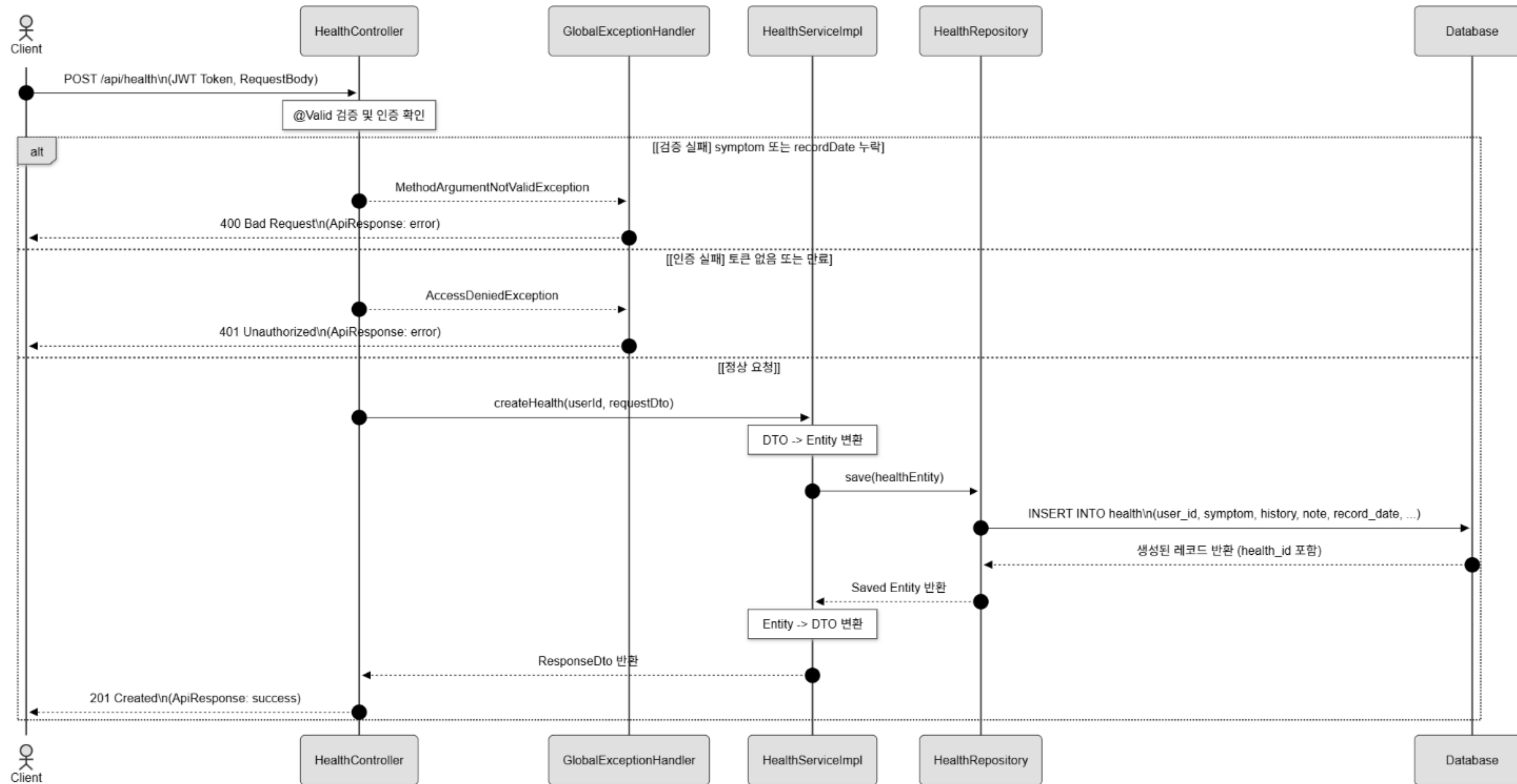
PUT /api/health/{health_id}

데이터 타입

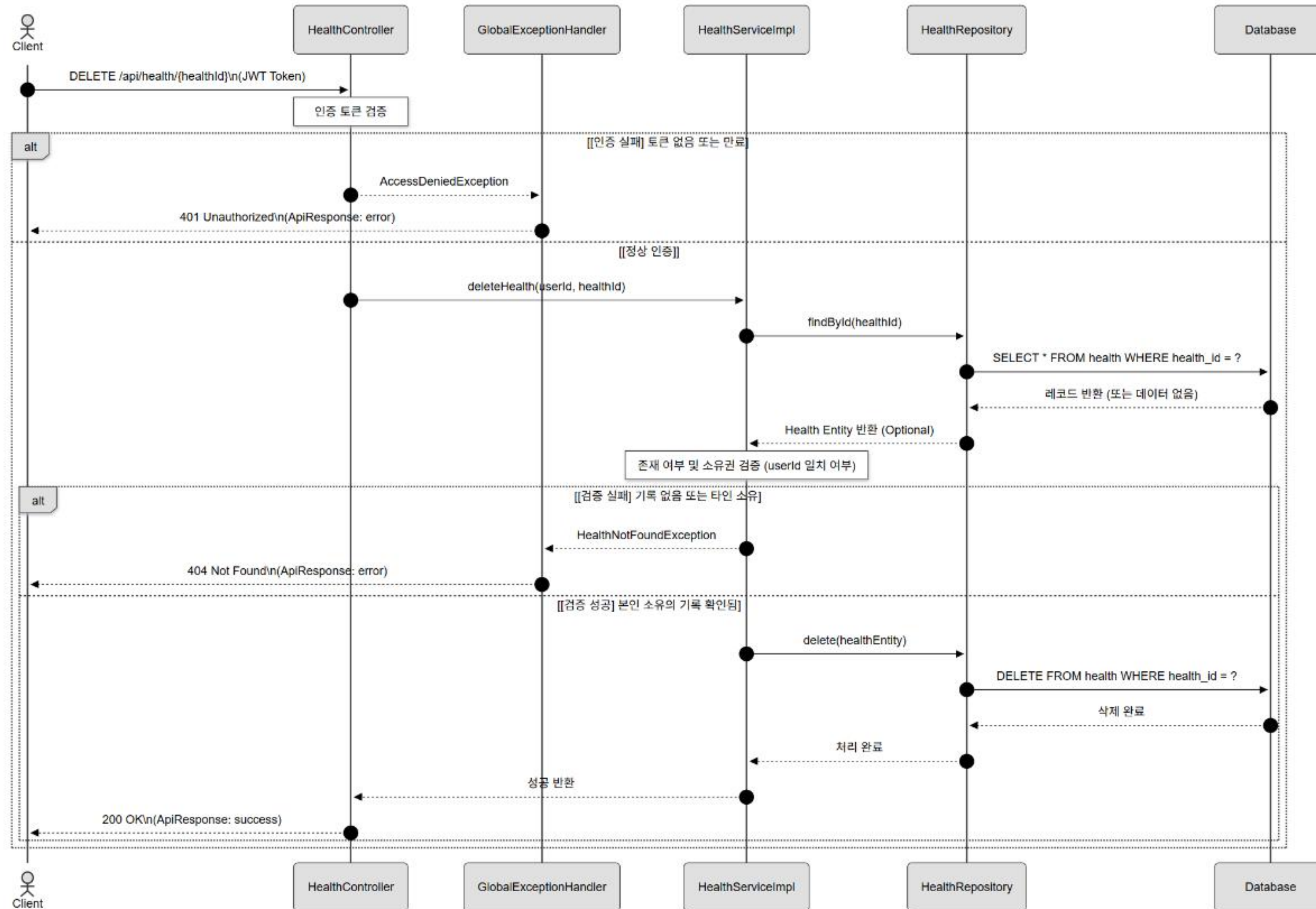
Request
recordDate : String
symptom : String
history : String
note : String

Response
status : int
message : String

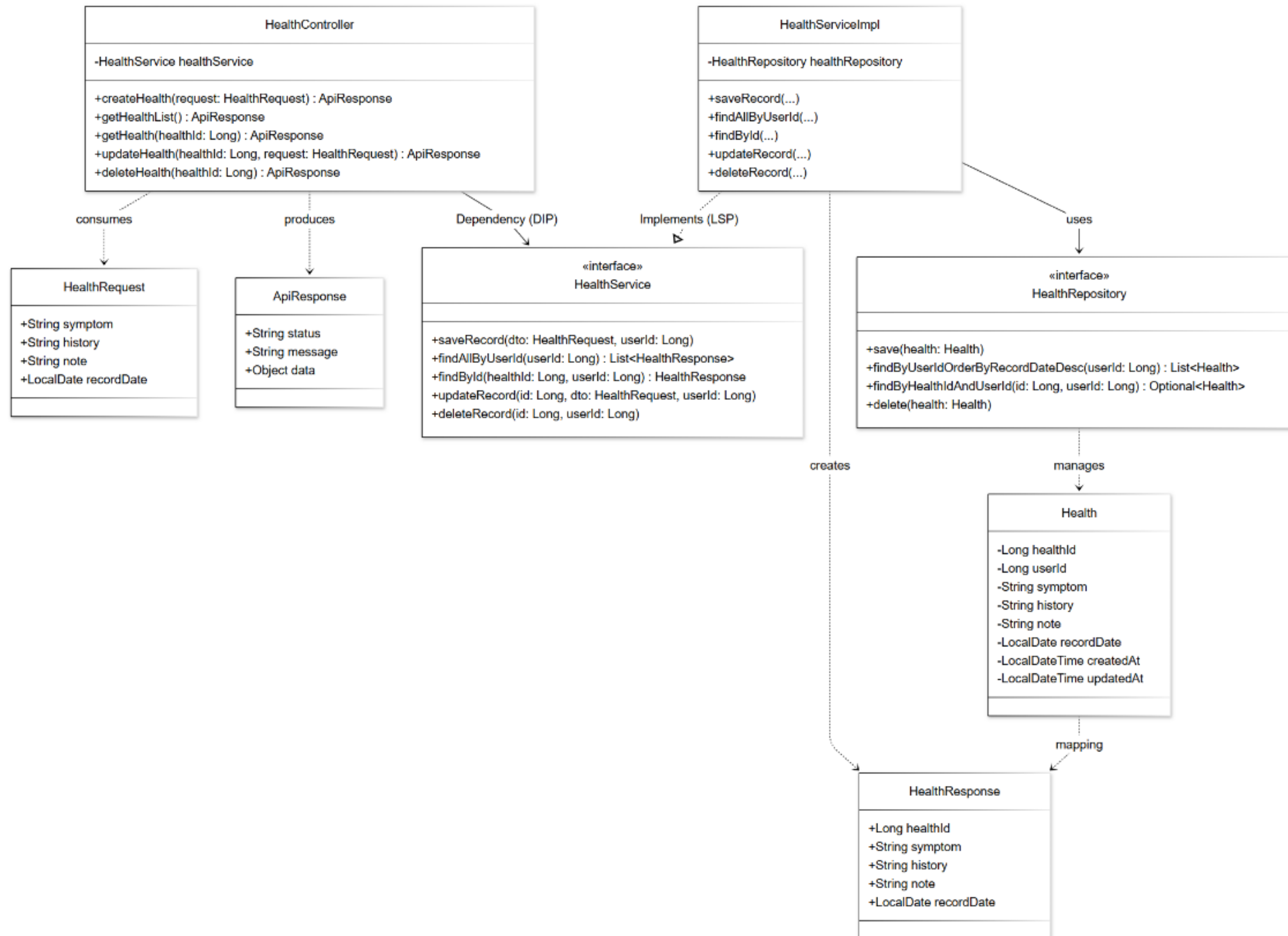
POST /api/health/# Sequence



DELETE /api/health/{health_id} Sequence

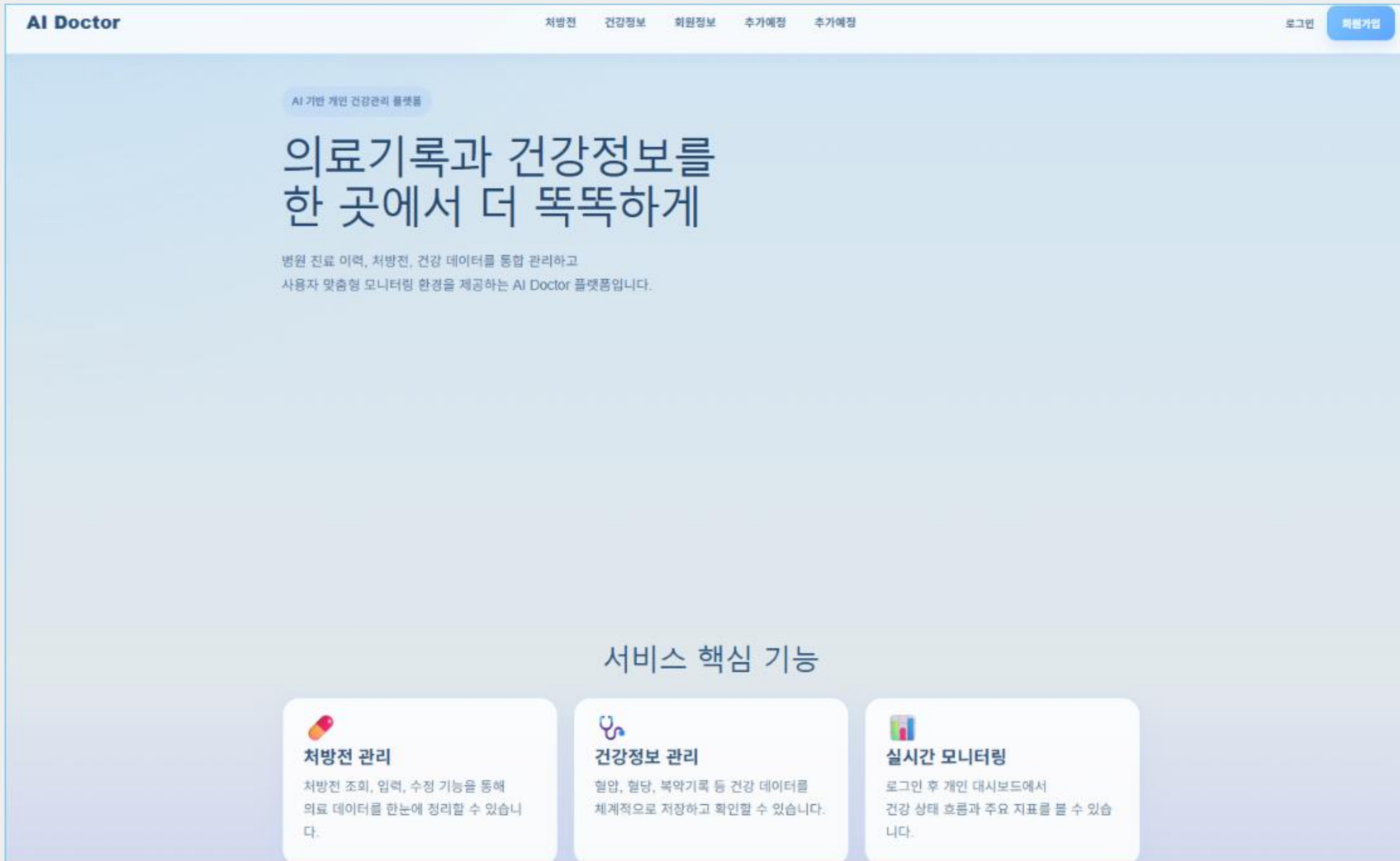


Health Domain Class Diagram



공통 UI/UX

MainPage - 비로그인 랜딩 화면



- 비로그인 사용자가 처음 접속했을 때 제공되는 랜딩 페이지.
 - 상단 네비게이션에 처방전.건강정보.회원정보 메뉴를 배치하여 서비스 구조 미리 파악 가능.
 - 로그인.회원가입 버튼을 통해 인증 화면으로 이동할 수 있다.
 - 하단 서비스 핵심 기능 카드: 처방전 관리.건강정보 관리.실시간 모니터링
- API 연동 # 정적 랜딩 페이지, API 호출 없음

데이터 타입

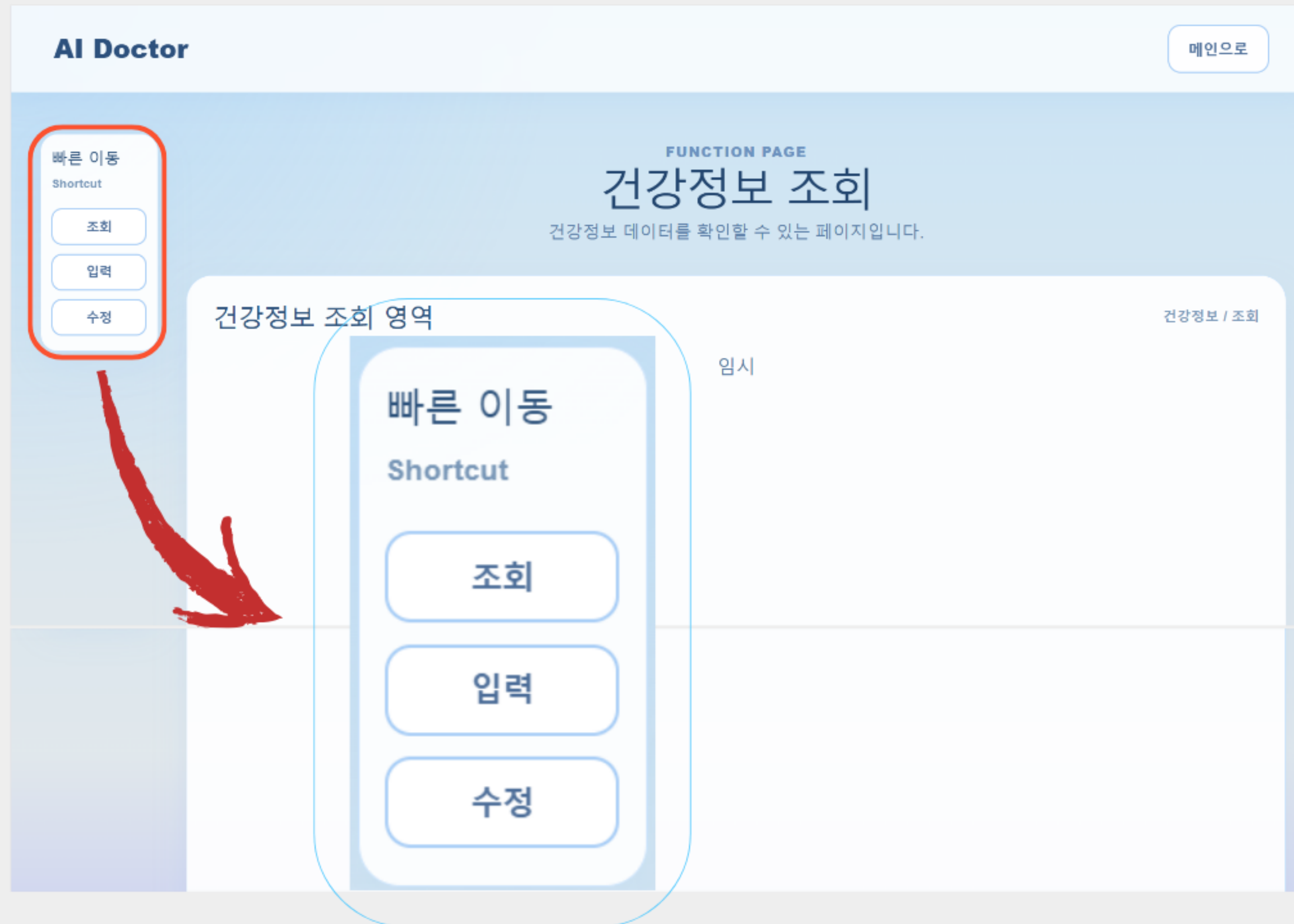
■ 컴포넌트 흐름

MainPage.tsx 진입

- 로그인 상태 확인
- 비로그인 시 랜딩 화면 표시
- 서비스 기능 카드 제공
- MainPage.tsx 유지

공통 UI/UX

빠른 이동 사이드바



- ManagePage 내 좌측 고정 패널로 빠른 화면 전환 지원
- category prop 에 따라 처방전 / 건강정보 메뉴 동적 렌더링
- activeMenu prop 으로 현재 활성 항목 하이라이트 처리
- 직접 API 호출 없이 useNavigate 혹은 기반 SPA 라우팅만 수

API 연동

직접 API 호출 없음 -> 클라이언트 라우팅만 처리

데이터 타입

category : prescription
-> 처방전 조회/입력/수정 링크

category : health
-> 건강정보 조회/입력/수정 링크

UI/UX

Health Domain UI - 건강 모니터링 대시보드



- 로그인 후 개인 건강 모니터링 화면
- 4개 요약 카드 + 건강 추이 그래프
- 데이터 최신화 버튼 -> LLM 분석 요청
- 쿨타임 1시간 제한 (중복 요청 방지)

API 연동

POST /api/ai/trigger

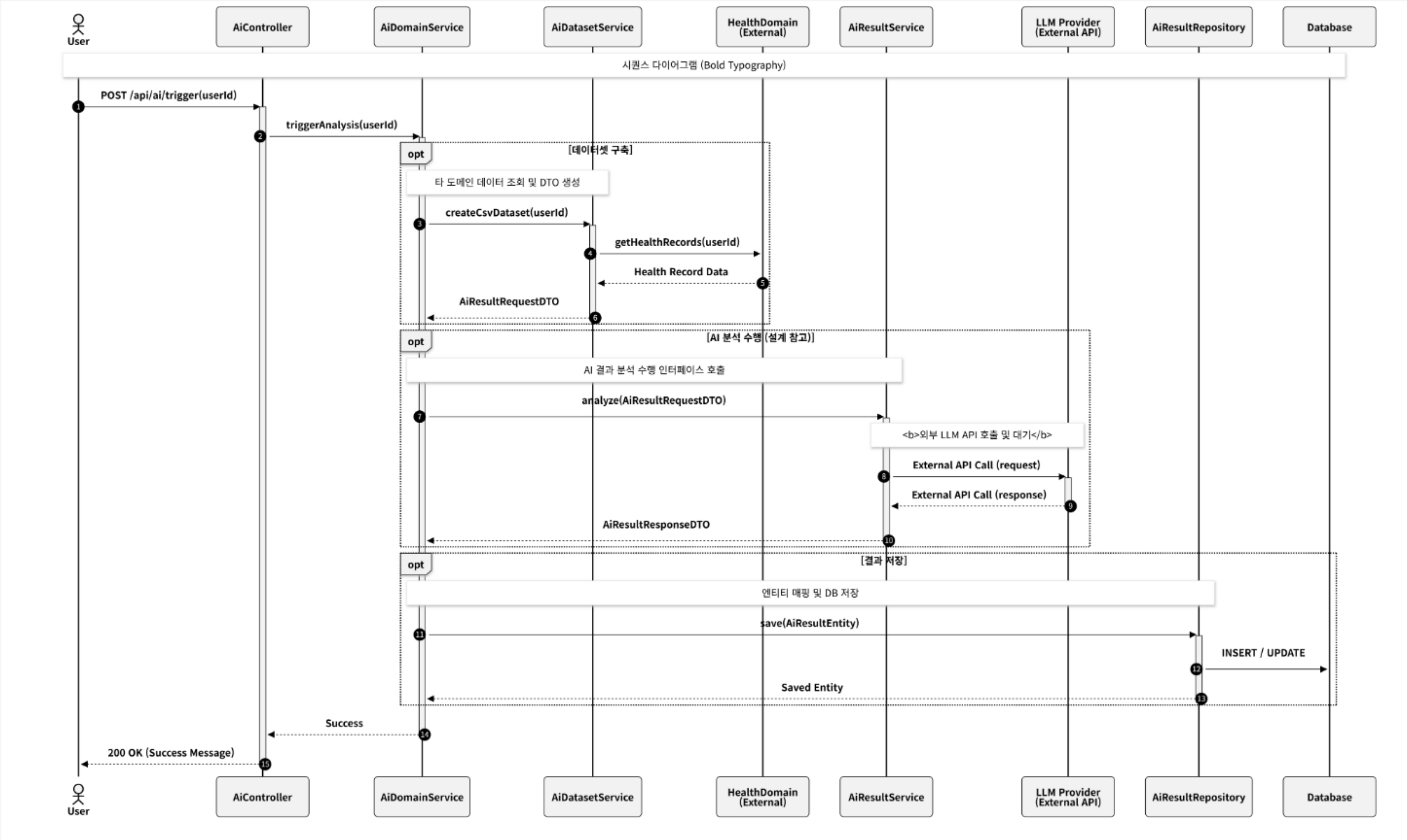
GET /api/ai/result/me

GET /api/ai/result/me/summary

데이터 타입

healthStatus : String
summaryNote : String
potentialDiseases : Array
recommendedFoods : Array
recommendedExercises : Array
precautions : String
analysisDate : String

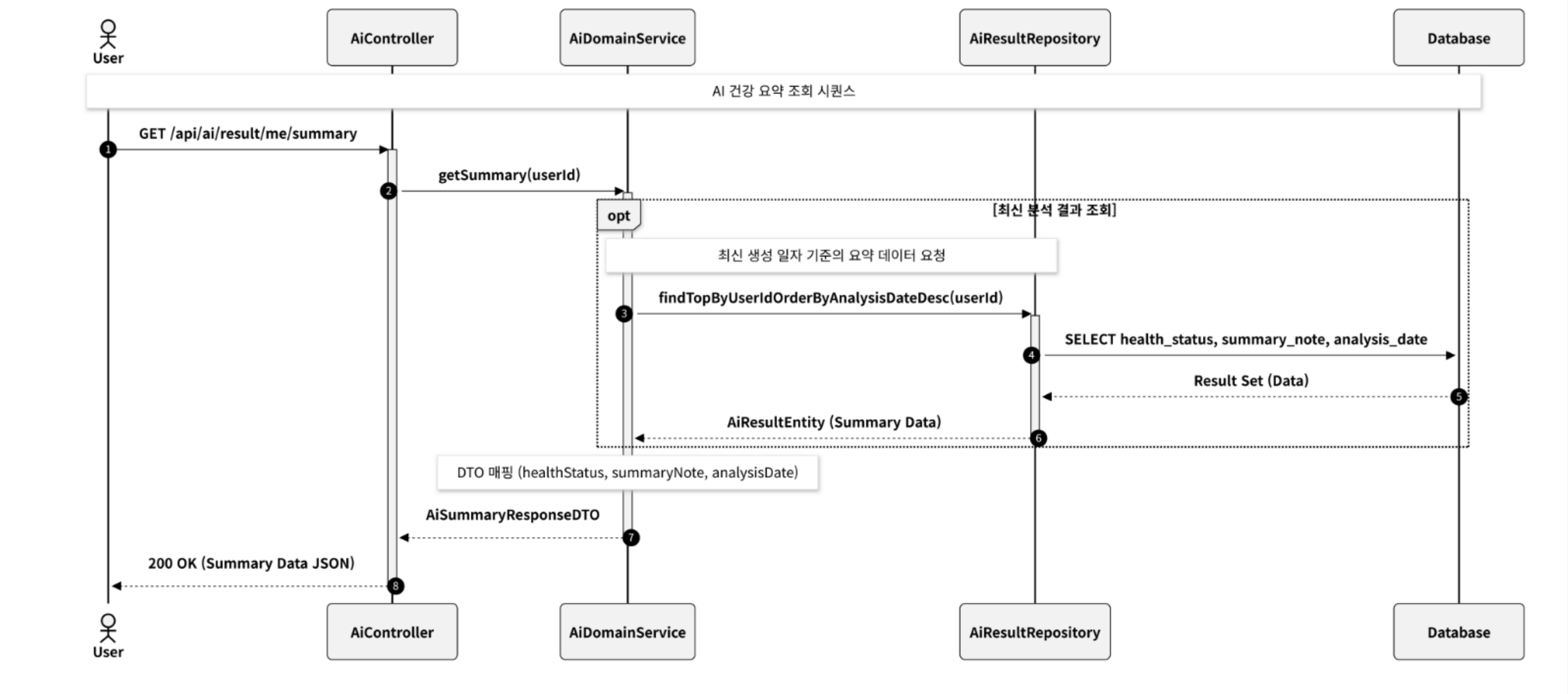
POST /api/ai/trigger Process(LLM API Pipeline)



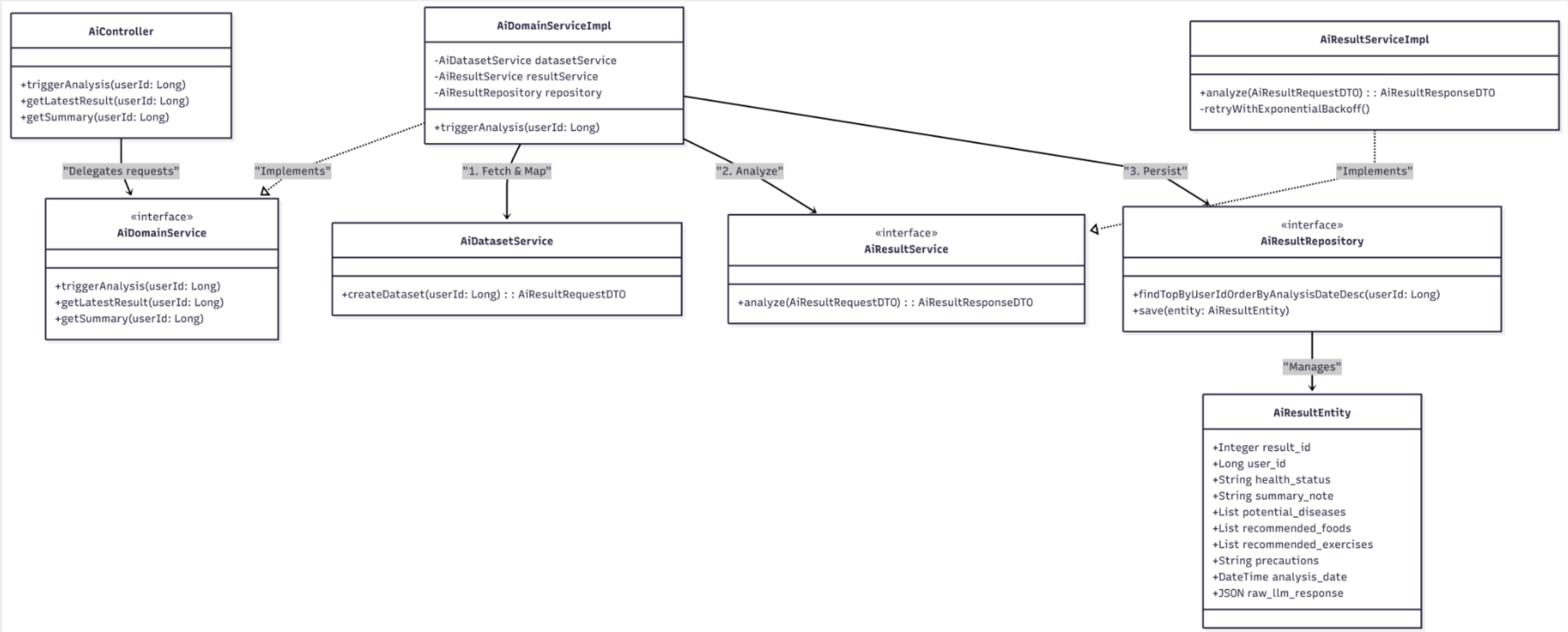
AI Sequence Diagram

GET

/api/ai/result/me/summary

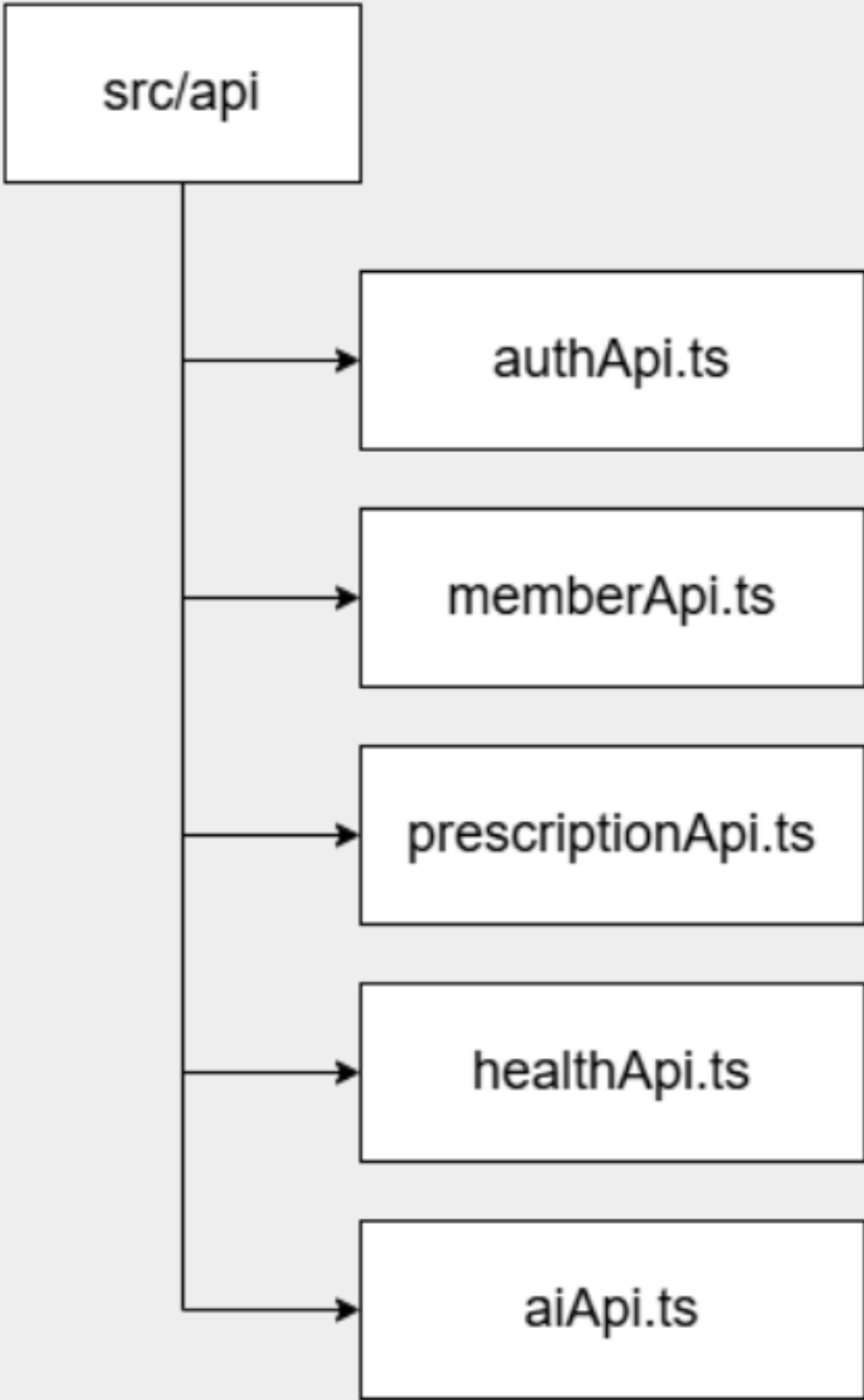


AI Domain Class Diagram



UI/UX

API Module - API 파일 분리 설계



- 프론트엔드에서는 화면 컴포넌트가 서버 요청 로직을 직접 포함하지 않고, 도메인별 API 모듈을 통해 백엔드와 통신하도록 설계함.

로그인 기능은 authApi를 통해 호출되며, 처방전, 건강정보, 회원정보, AI 분석 기능도 각각 별도의 API 모듈로 분리되었다.

백엔드 API 주소나 요청 형식이 변경되더라도 관련 API 모듈만 수정하면 되도록 하였음.

화면	API 모듈	역할
LoginPage	authApi	로그인, 토큰 저장
SignupPage	authApi/memberApi	회원가입
ManagePage	prescriptionApi	처방전 CRUD
ManagePage	healthApi	건강정보 CRUD
MainPage Dashboard	aiApi	AI 분석 결과 조회

UI/UX

Domain mapping - UI + API + Backend 연결

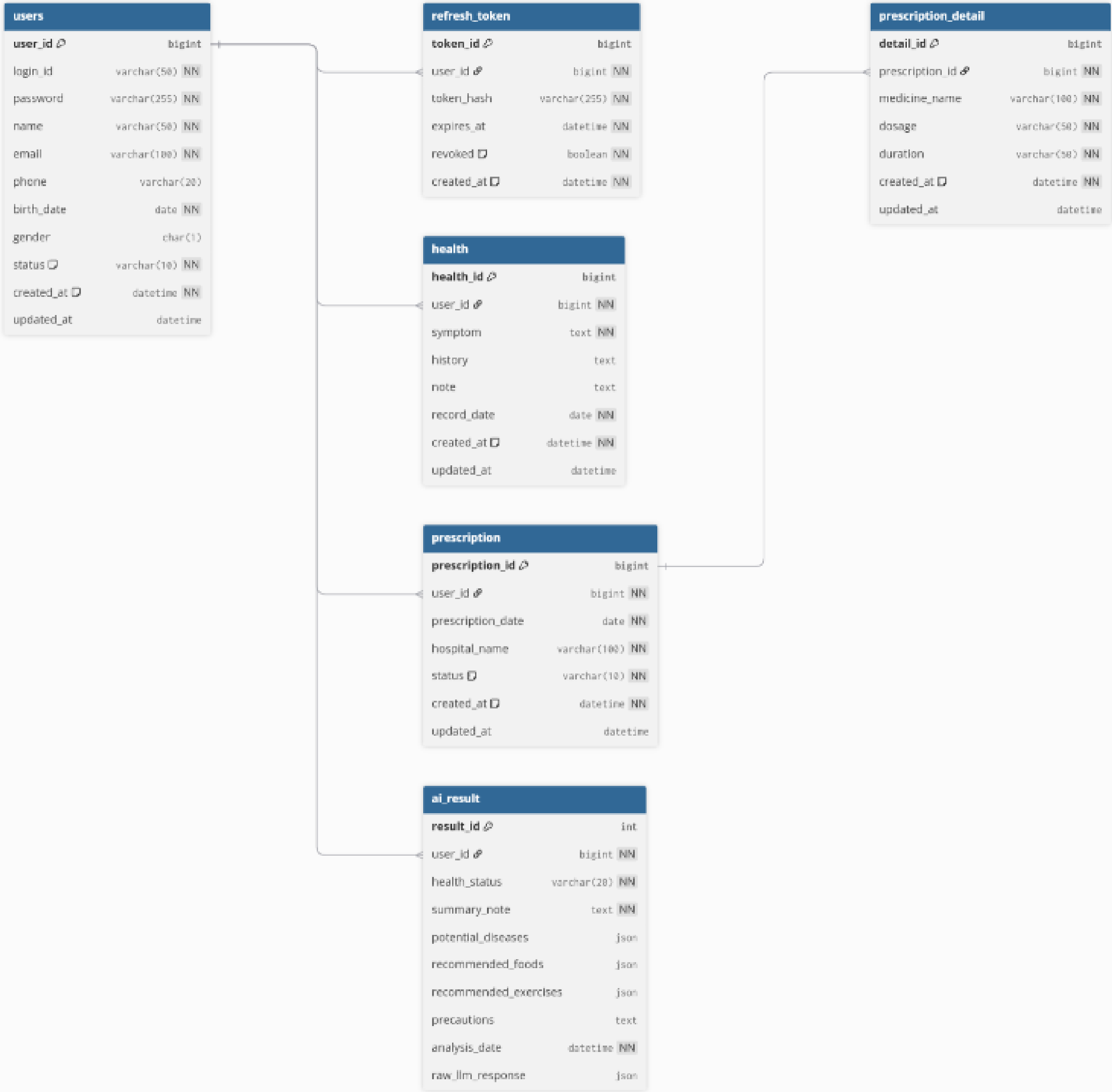
UI 화면	프론트 컴포넌트	API 모듈	백엔드 도메인
로그인	LoginPage	authApi	Auth / Member
회원가입	SignupPage	authApi/memberApi	Auth / Member
메인 랜딩	MainPage	-	서비스 소개
대시보드	MainPage	aiApi	AI / Health / Prescription
처방전 관리	ManagePage	prescriptionApi	Prescription
건강정보 관리	ManagePage	healthApi	Health

ERD Diagram

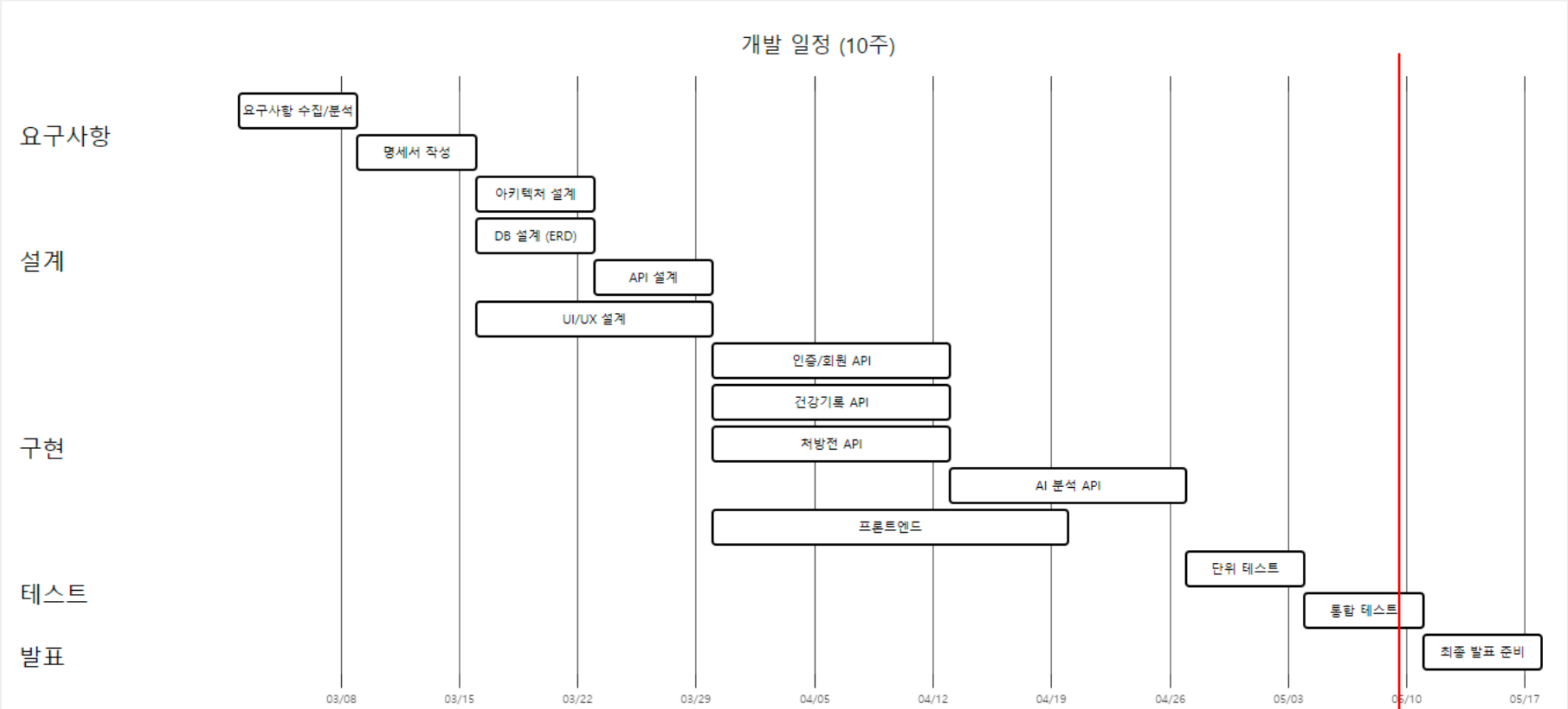
사용자 관리 도메인
users / refresh_token

의료/건강 데이터 도메인
health / prescription/prescription_detail

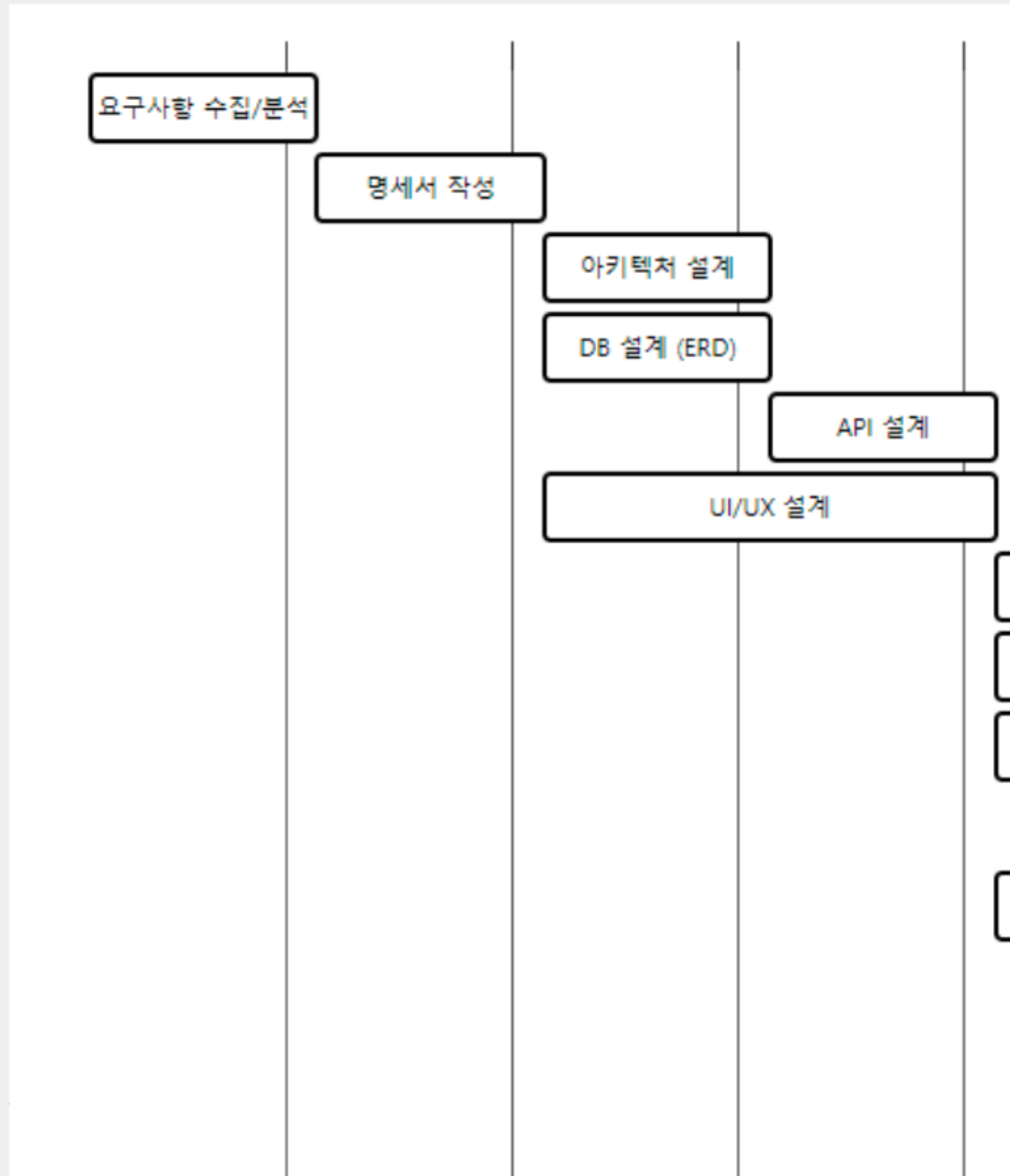
AI 서비스 도메인:
ai_result



전체 개발 일정



1 - 4주차



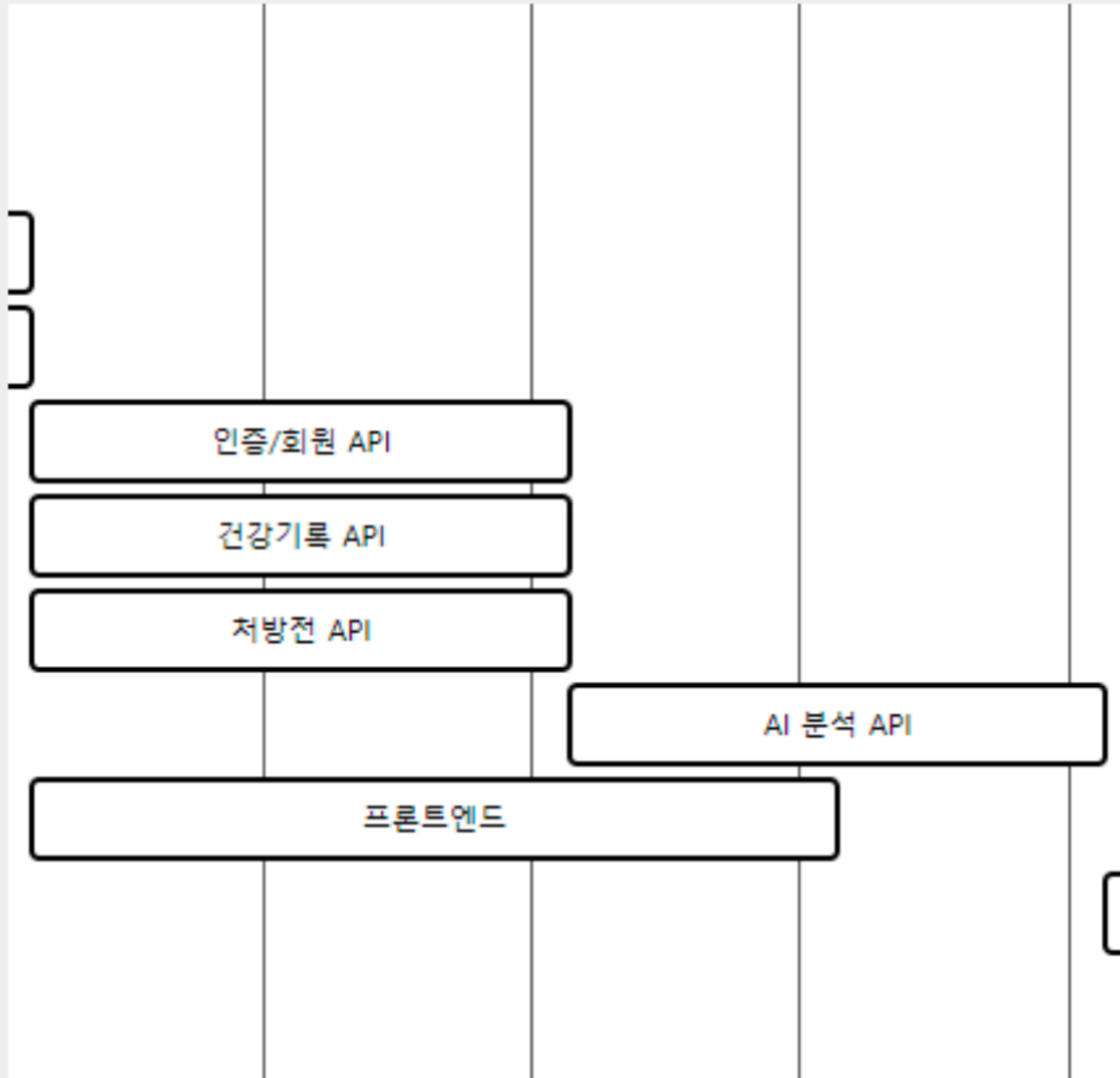
● 1 ~ 2주차 (요구 분석 단계)

- 서비스의 전반적인 목표와 범위 확정
- 프로젝트의 방향성 구체화
- 주요 기능 리스트업을 통해 시스템의 핵심 요구사항 확인
- 팀 내 담당 파트 분담

● 3 ~ 4주차 (프로젝트 구현 단계)

- 요구 분석 결과 바탕, 시스템 아키텍처 및 DB 모델링
- 프론트엔드의 UI 시안 설계
- 백엔드 API 명세서 작성
- 개발 환경 세팅

5 - 8주차



● 5 ~ 8주차 (프로젝트 구현 단계)

○백엔드:

- 회원가입, 로그인, 건강 기록·처방전 관리, AI 분석 요청 기능 구현
- LLM 기반 비즈니스 로직 연동

○프론트엔드:

- 사용자 UI 개선
- 페이지 간 데이터 연동
- AI 분석 대시보드 화면 구현

9 - 10주차



- 9주차 (테스트 및 배포 단계)
 - 단위 및 통합 테스트 수행
 - 버그 수정
- 10주차 (발표 준비 및 발표)
 - 코드, 문서 최종 정리
 - 제출

추후 대응

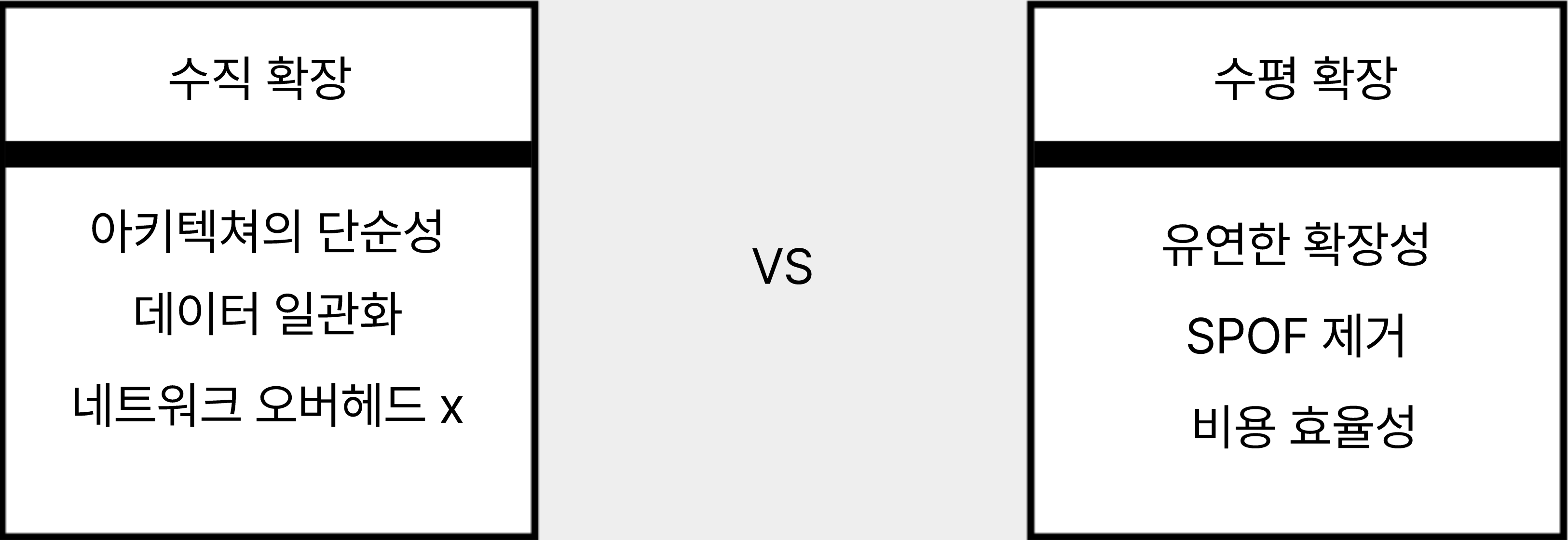
추후대응

시스템 확장의 2가지 방법

수직확장(Scale Up) : 서버 자체의 스펙을 선형적 증가

수평확장(Scale Out) : 노드를 추가하여 병렬 분산처리

1차적 비교분석



추후 대응 - TradeOff

왜 어떤 확장이 본 프로젝트에 적합한가?

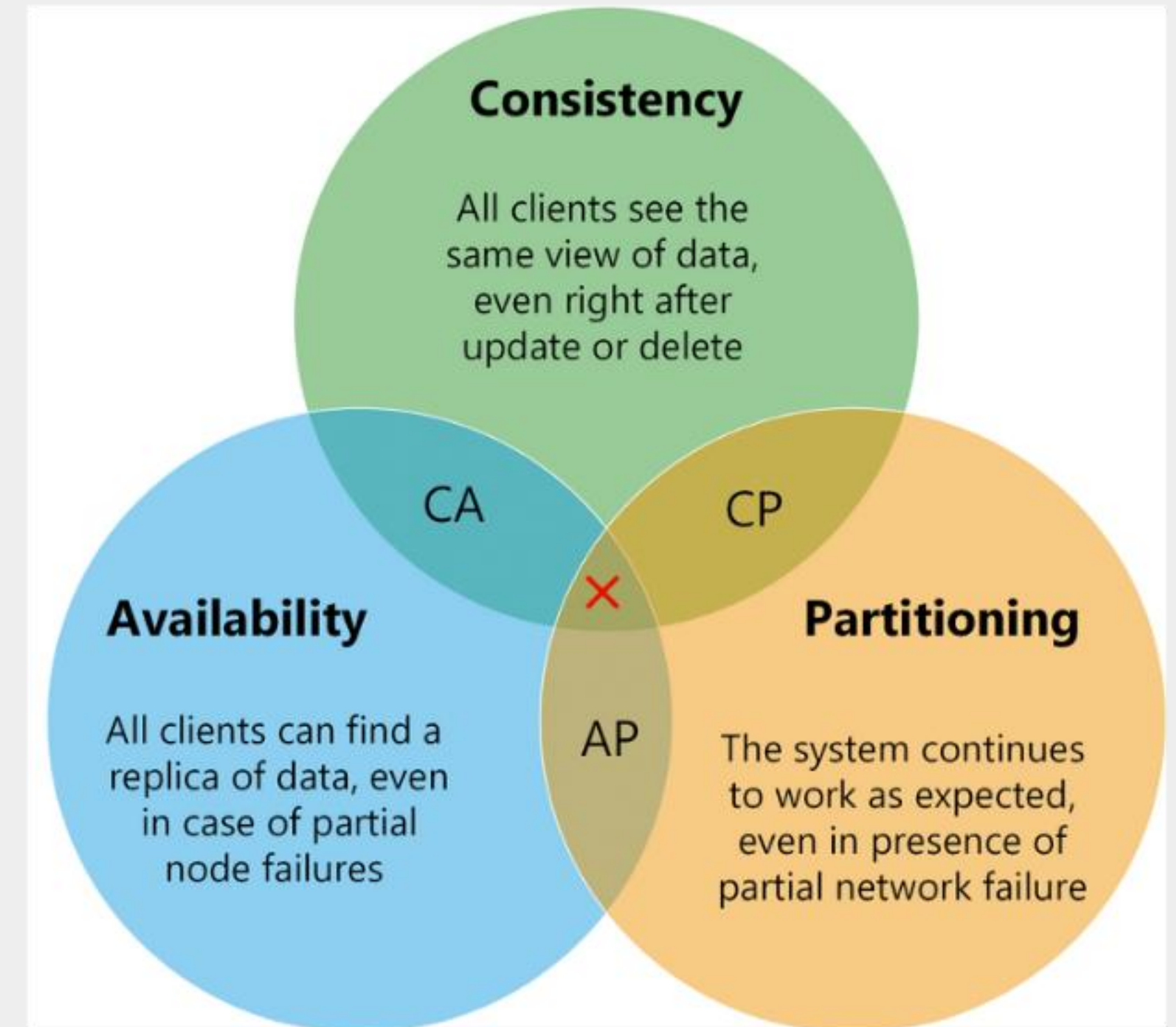
대부분의 서버 확장은 수평확장을 통한 분산 처리에 의해 이루어짐

수직확장에는 한계가 명확하며 단점이 많음.

비용이 수직으로 치솟으며 분산시스템의 3요소 중 하나인 가용성 자체가 성립되지 못하며 그 자체가 SPOF가 됨.

수직확장의 경우 일관성이 극도로 중요한 시스템에 필요한 구조.

하지만 본 프로젝트 처럼 소규모의 경우 분산 처리를 위한 로드밸런싱의 비용과 관리 복잡도를 고려했을때 충분히 트레이드 오프 여지가 존재.

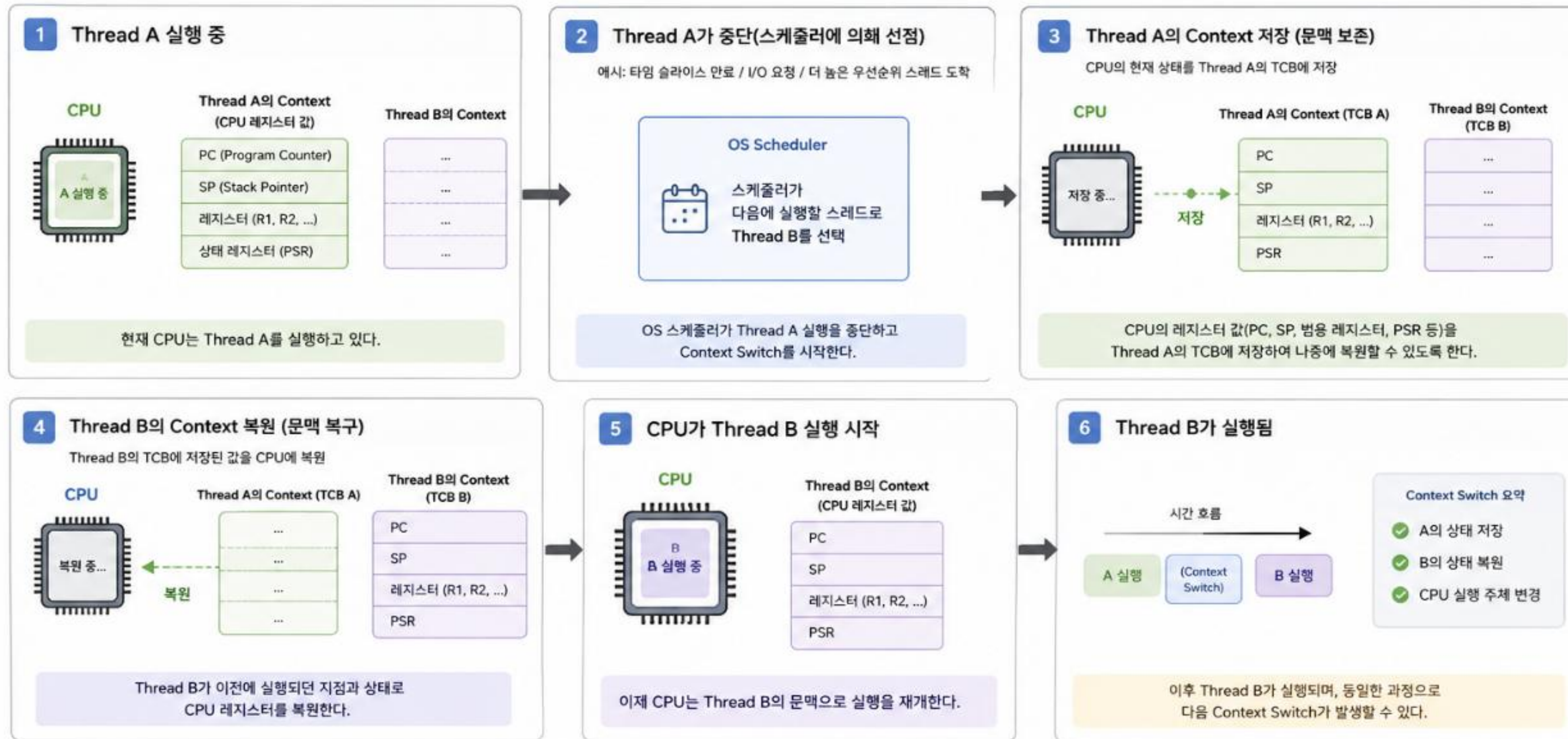


추후 대응

Thread Context Switching

추후대응

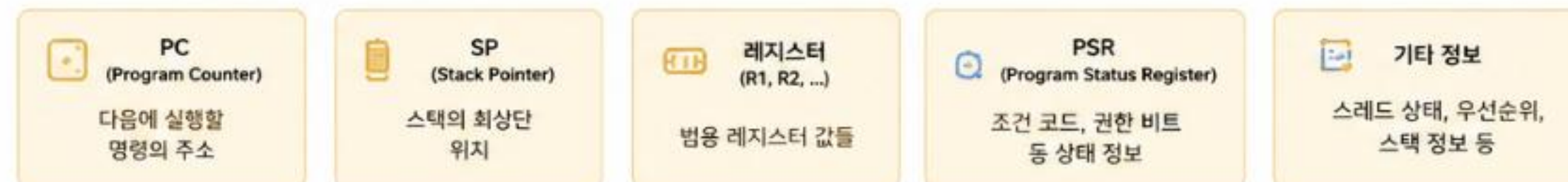
Thread Context Switching 이 일어나는 과정



💡 핵심 포인트

- Context Switch는 CPU가 한 스레드에서 다른 스레드로 전환할 때 발생한다.
- 현재 스레드의 상태(문맥)를 저장하고, 다음 스레드의 상태를 복원해야 한다.
- 이 과정에서 CPU는 잠시 다른 일을 하지만, 각 스레드는 이전에 멈췄던 지점부터 이어서 실행할 수 있다.

TCB (Thread Control Block)에 저장되는 주요 정보 예시



서버를 수직확장한다고 해서 스레드를 무한정 늘릴 수 없다.
스레드의 점유가 주 병목인 본 프로젝트 특성상 추가 확장 필요시 로드밸런싱과 데이터 일관성 등의 커브를 감수하고 수평확장을 도입하는 것이 올바른 설계이다.

추가로 고려해볼 것 : 논블로킹큐

이상으로 발표를 마치겠습니다.
